

Large Language Models

Dr. Jingjing Deng
Senior Lecturer in AI, SEMT
University of Bristol, UK



- 1980s - Rediscover Back-propagation
- 1990s - Winter for Neural Networks
 - LeNet Handwritten Digits Recognition
- 2000s - Era of Internet & Gaming
 - Massive Data & GPGPUs
- 2010s - Early Success & Boom of ANN
 - AlexNet (CNN) on ImageNet
 - LSTM & Transformers (Attention)
- 2024 - Nobel Prize Winners
 - Physics - Artificial Neural Networks
 - Chemistry - AlphaFold Protein Design

The Nobel Prize in Physics 2024

John J. Hopfield

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”



John Hopfield. Ill. Niklas Elmehed © Nobel Prize Outreach

Geoffrey Hinton

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”



Geoffrey Hinton. Ill. Niklas Elmehed © Nobel Prize Outreach

The Nobel Prize in Chemistry 2024

David Baker

“for computational protein design”



David Baker. Ill. Niklas Elmehed © Nobel Prize Outreach

Demis Hassabis

“for protein structure prediction”



Demis Hassabis. Ill. Niklas Elmehed © Nobel Prize Outreach

John Jumper

“for protein structure prediction”

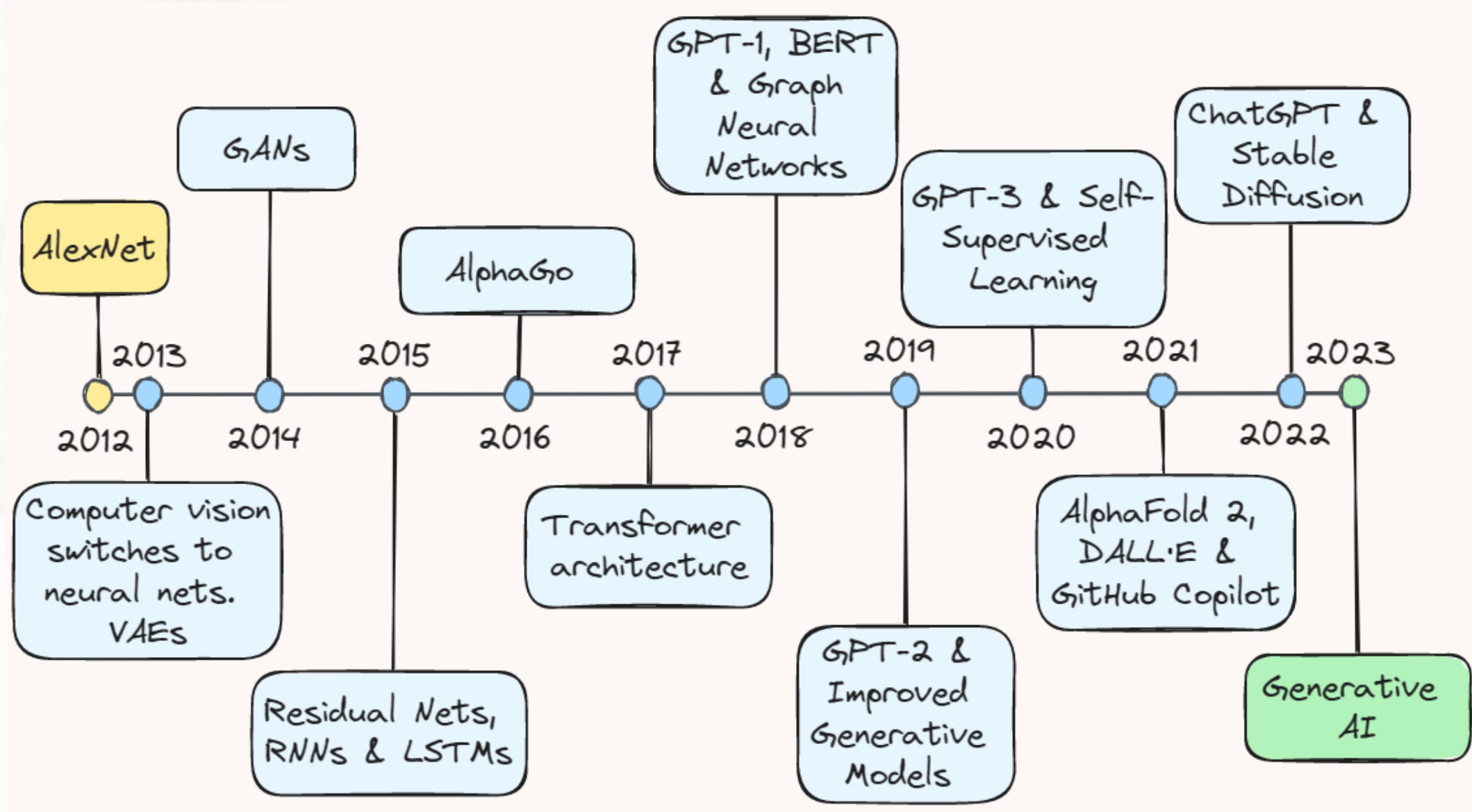


John Jumper. Ill. Niklas Elmehed © Nobel Prize Outreach

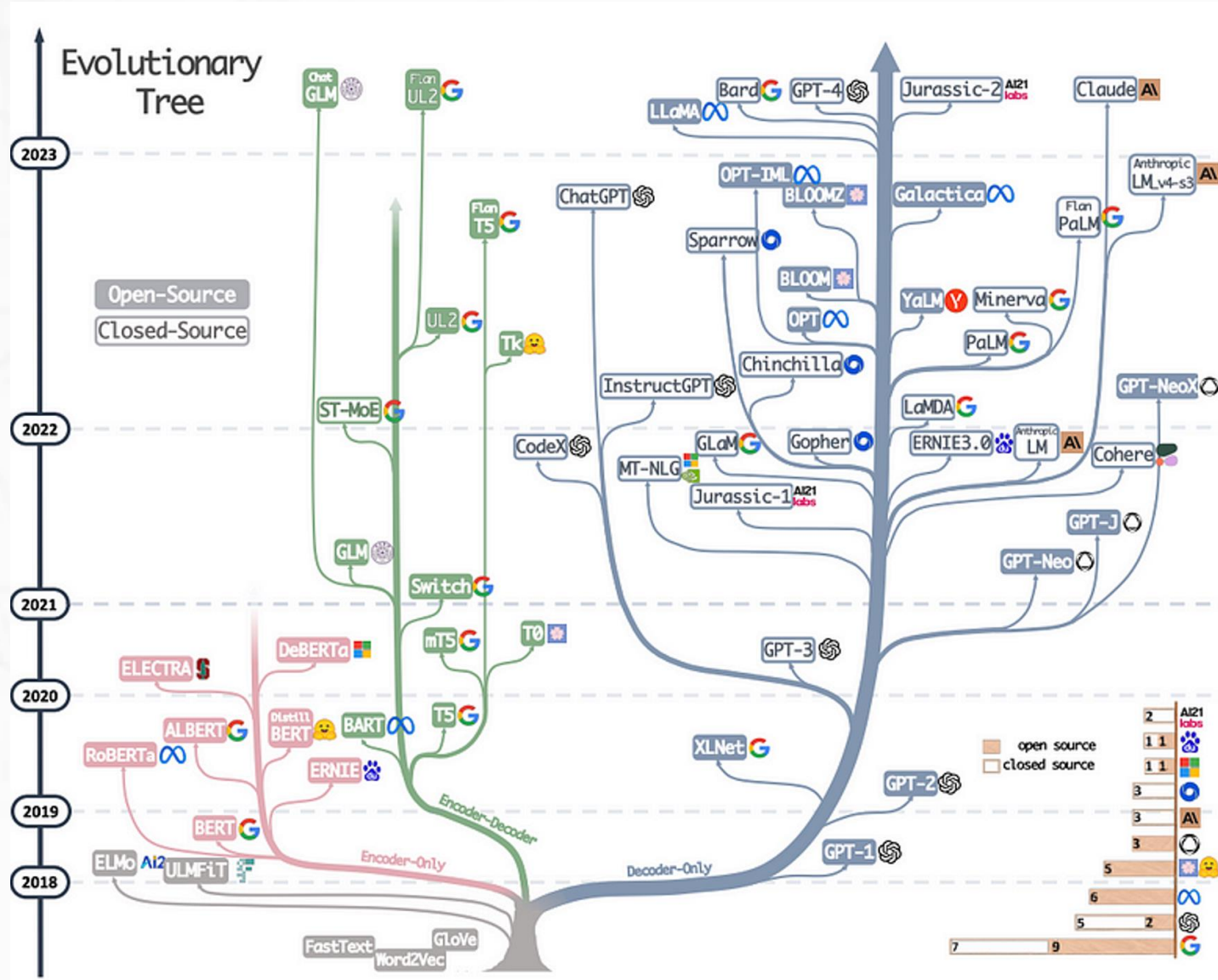
1. Rumelhart, D.E., Hinton, G.E. and Williams, R.J., 1986. Learning representations by back-propagating errors. nature, 323(6088), pp.533-536.

2. <https://www.nobelprize.org/all-nobel-prizes-2024/>

“Deep” Learning Driven AI Era



An Incomplete Map of LLMs



1. <http://keg.cs.tsinghua.edu.cn/jietang/>

2. <https://blog.sylphai.com/introduction-to-large-language-models>

Representation of Words

Tokenizer & Embedding

Tokenization

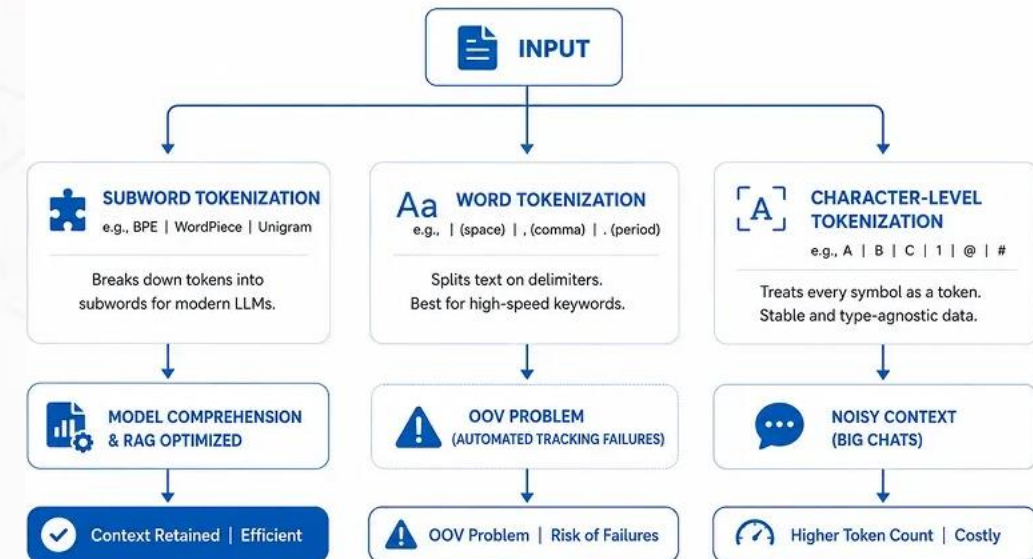
- A tokenizer is a foundational tool in Artificial Intelligence and Natural Language Processing (NLP). It acts as a bridge between human language and machine learning models.
- AI cannot "read" letters or words directly, a tokenizer breaks down raw text into smaller, manageable chunks called tokens and assigns each token a unique numerical ID.
- Subword: BPE – GPT/Llama, WordPiece – BERT, Unigram – Google T5

"This is the first step in the NLP pipeline"

Tokenizer

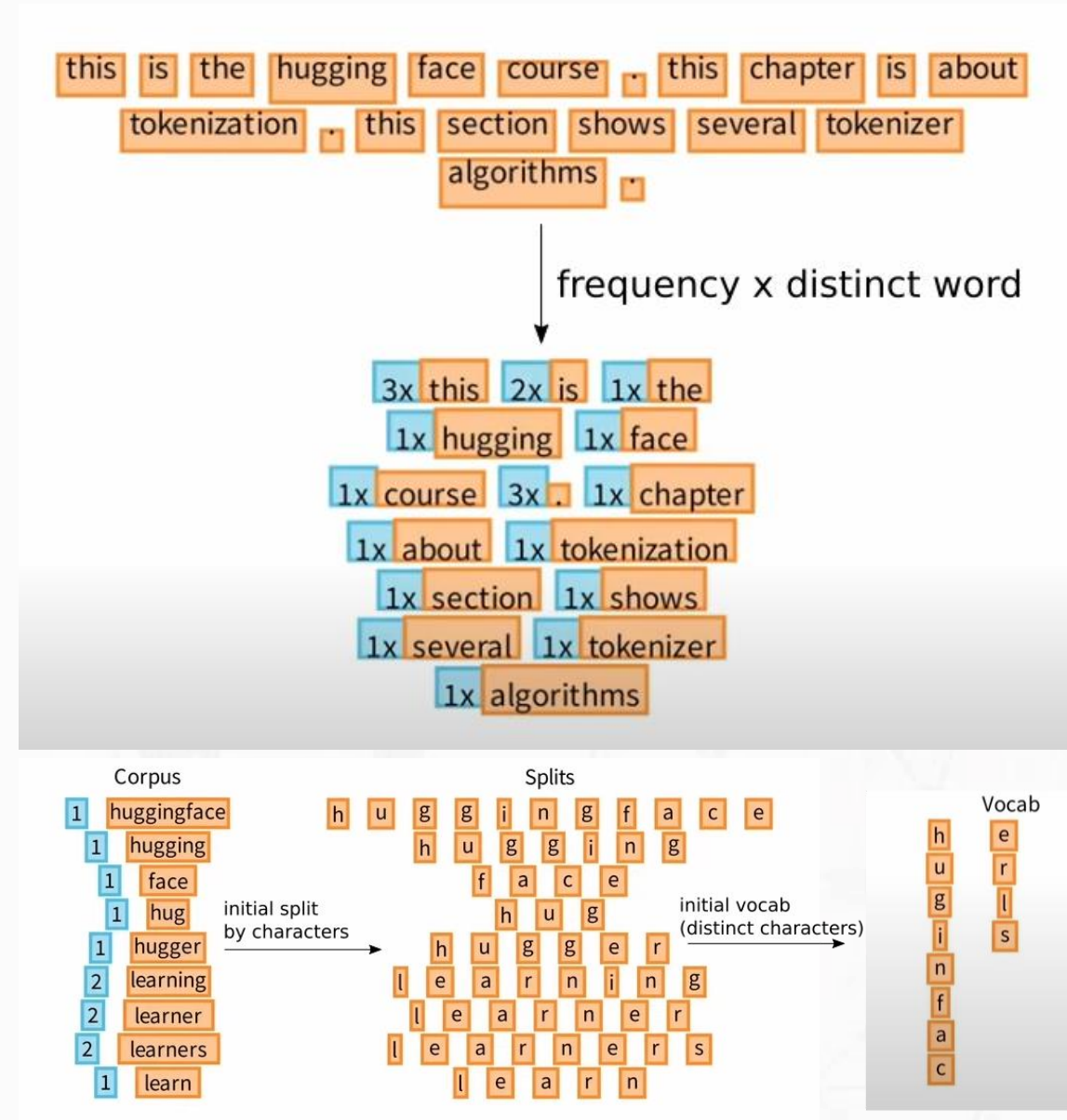
'This' 'is' 'the' 'first' 'step' 'in' 'the' 'NLP' 'pipeline'

The Modern Hierarchy of Tokenization Types

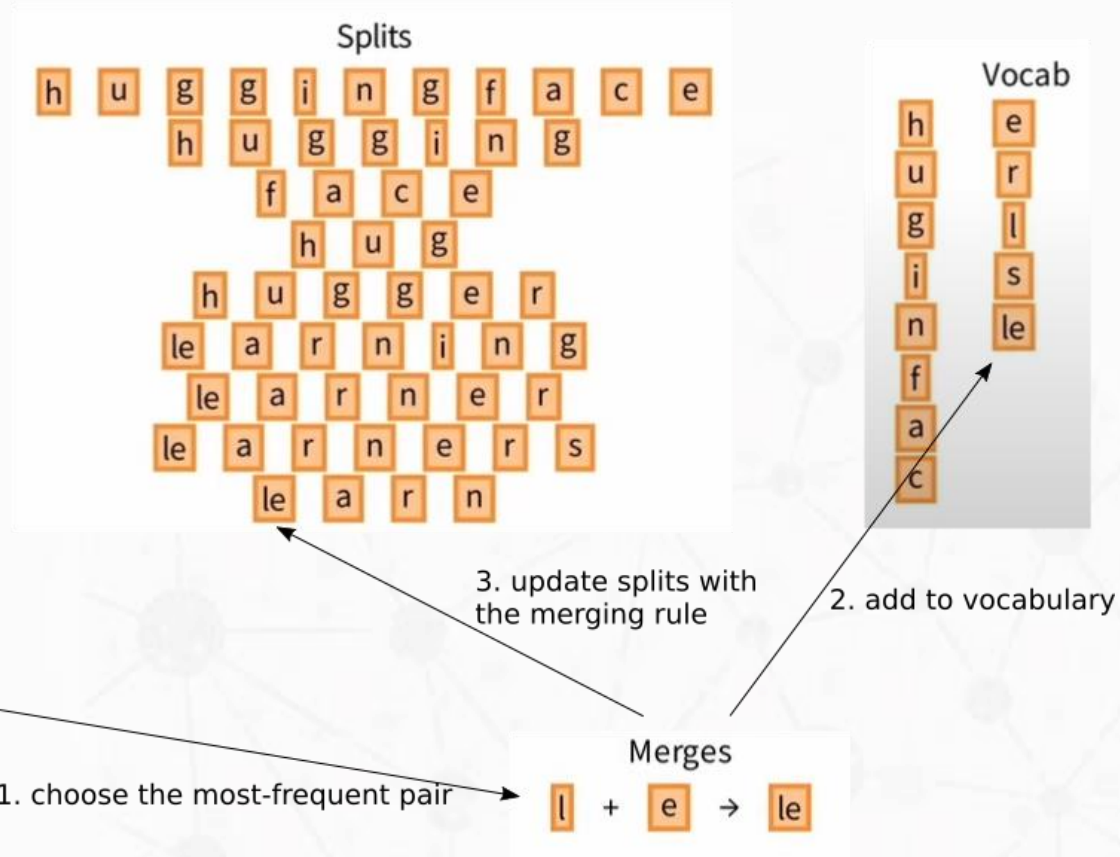
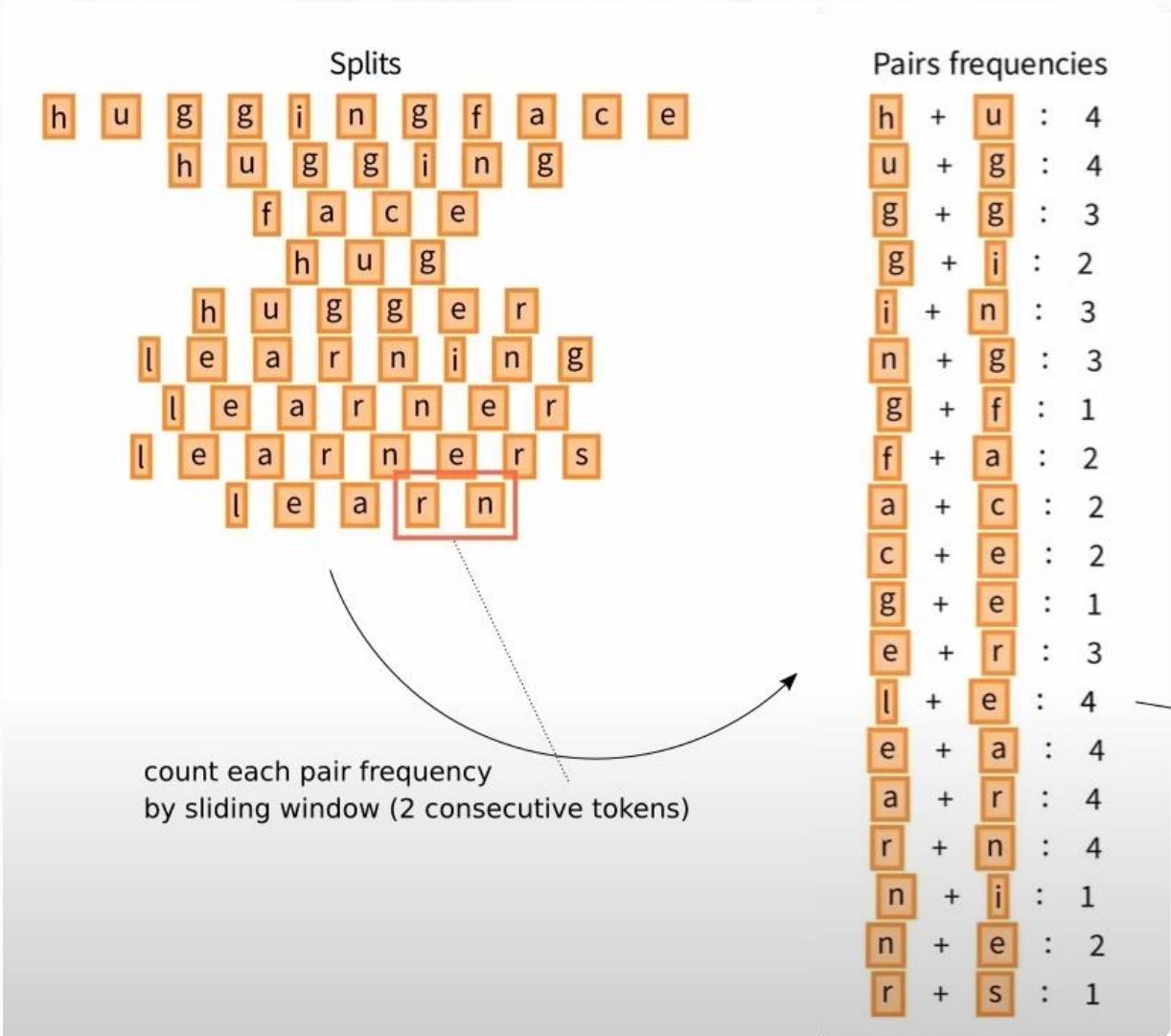


Byte-Pair Encoding (BPE)

- BPE is a subword tokenization algorithm that, after training, breaks text into smaller pieces called tokens.
 - “hugs” → (trained) BPE → “hug”, “s”
- Training a tokenizer is a statistical process that tries to identify which subwords are the best to pick for a given corpus, and the exact rules used to pick them depend on the tokenization algorithm.
- BPE training starts with a standardized and pre-tokenized corpus.

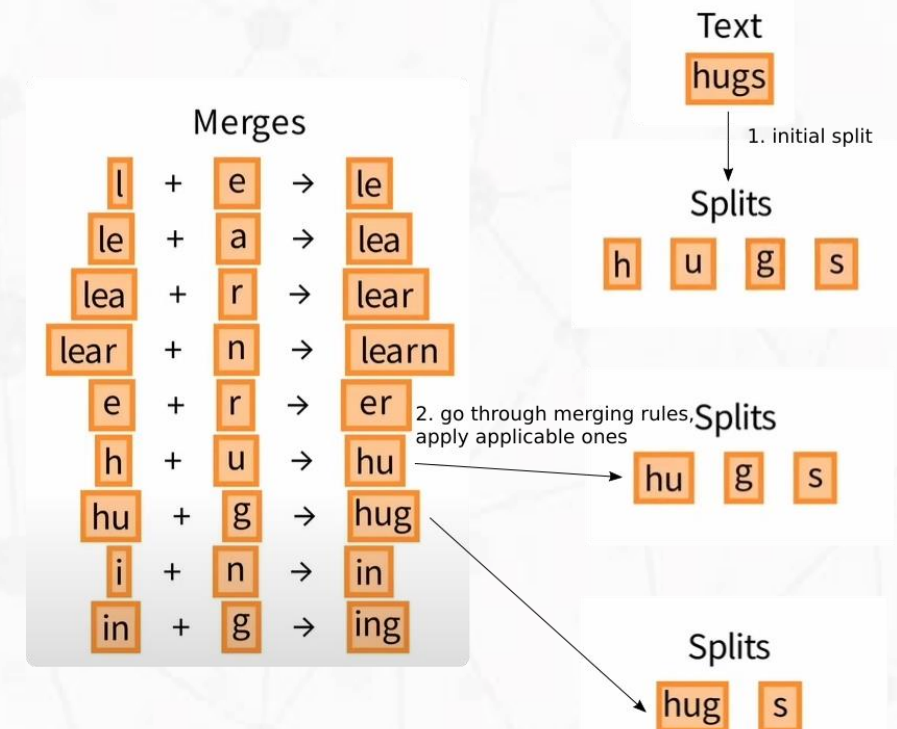
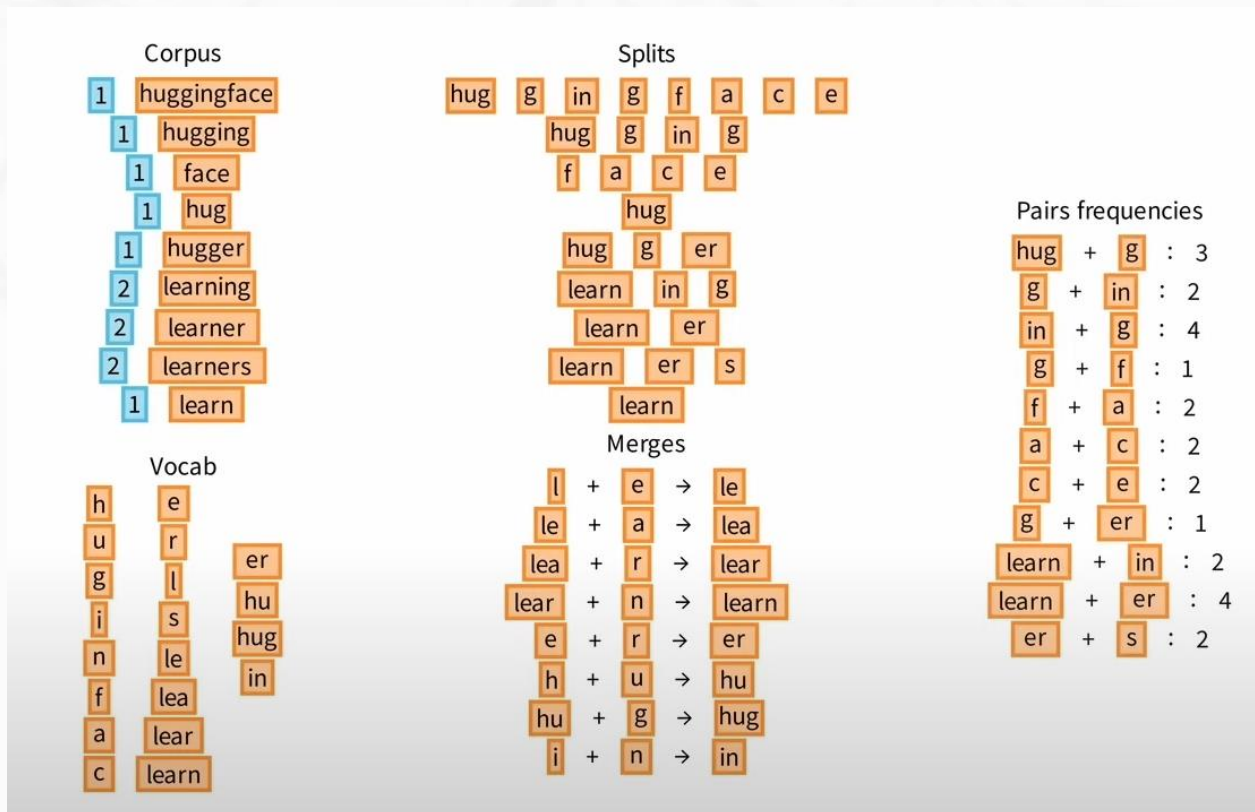


Pair Frequencies & Merging Rule



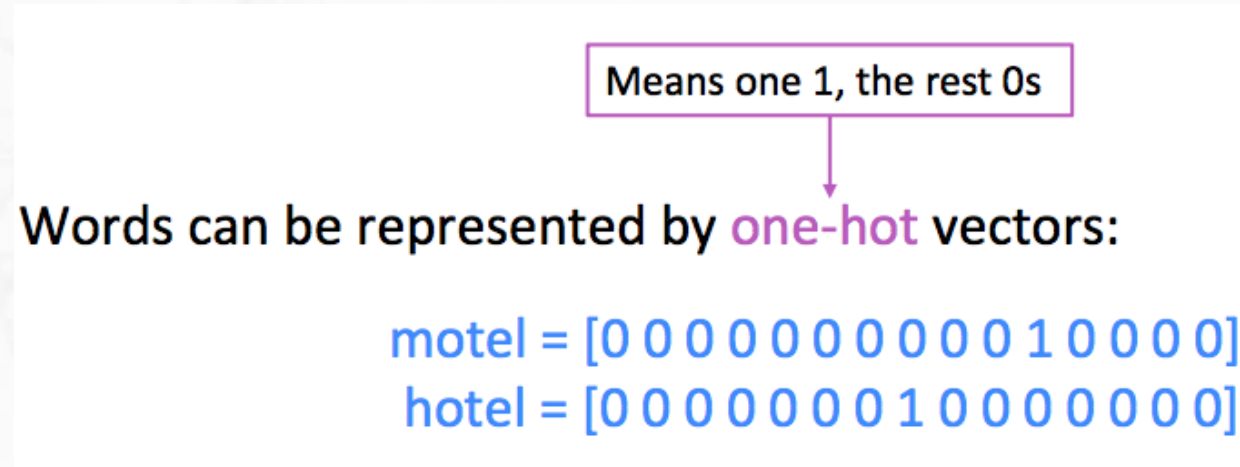
BPE Tokenizer - Training & Inference

- BPE training starts with the initial vocabulary and gradually expands it to reach the desired vocabulary size.
- BPE inference start by splitting the text into individual characters, then apply the learned BPE merging rules sequentially.



One-Hot Vector

- Vector dimension = number of words in vocabulary (e.g. 500,000)



- Limitation of One-Hot Vector
 - Very high dimensional and extremely sparse vector space.
 - These two vectors are orthogonal.
 - There is no natural notion of similarity for one-hot vectors.
 - i.e. Swansea motel and Swansea hotel cannot match.

Ideal Word Representation

- Represent meaning of word with natural notion of similarity.
- Low dimensional but compact vector space.
- Core idea: A word's meaning is given by the words that frequently appear close-by
 - “You shall know a word by the company it keeps” (J. R. Firth 1957: 11)
 - When a word w appears in a text, its context is the set of words that appear nearby (within a fixed-size window).
 - Use the many contexts of word to build up a representation of word.

*...government debt problems turning into **banking** crises as happened in 2009...*

*...saying that Europe needs unified **banking** regulation to replace the hodgepodge... **banking** =*

*...India has just given its **banking** system a shot in the arm...*

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Window based Co-occurrence Matrix

- As our first attempt, we make the bold conjecture that words that are related will often appear in the same documents.
 - For instance, "banks", "bonds", "stocks", "money", etc. are probably likely to appear together.
 - But "banks", "octopus", "banana", and "hockey" would probably not consistently appear together.

The resulting counts matrix will then be:

1. I enjoy flying.
2. I like NLP.
3. I like deep learning.

$X =$

	<i>I</i>	<i>like</i>	<i>enjoy</i>	<i>deep</i>	<i>learning</i>	<i>NLP</i>	<i>flying</i>	<i>.</i>
<i>I</i>	0	2	1	0	0	0	0	0
<i>like</i>	2	0	0	1	0	1	0	0
<i>enjoy</i>	1	0	0	0	0	0	1	0
<i>deep</i>	0	1	0	0	1	0	0	0
<i>learning</i>	0	0	0	1	0	0	0	1
<i>NLP</i>	0	1	0	0	0	0	0	1
<i>flying</i>	0	0	1	0	0	0	0	1
<i>.</i>	0	0	0	0	1	1	1	0

$= \frac{1}{|V|} \begin{bmatrix} | & | & \dots & | \\ u_1 & u_2 & & \\ | & | & & | \end{bmatrix}^k \begin{bmatrix} \sigma_1 & 0 & \dots \\ 0 & \sigma_2 & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}^k \begin{bmatrix} - & v_1 & - \\ - & v_2 & - \\ \vdots & \vdots & \end{bmatrix}^{|V|}$

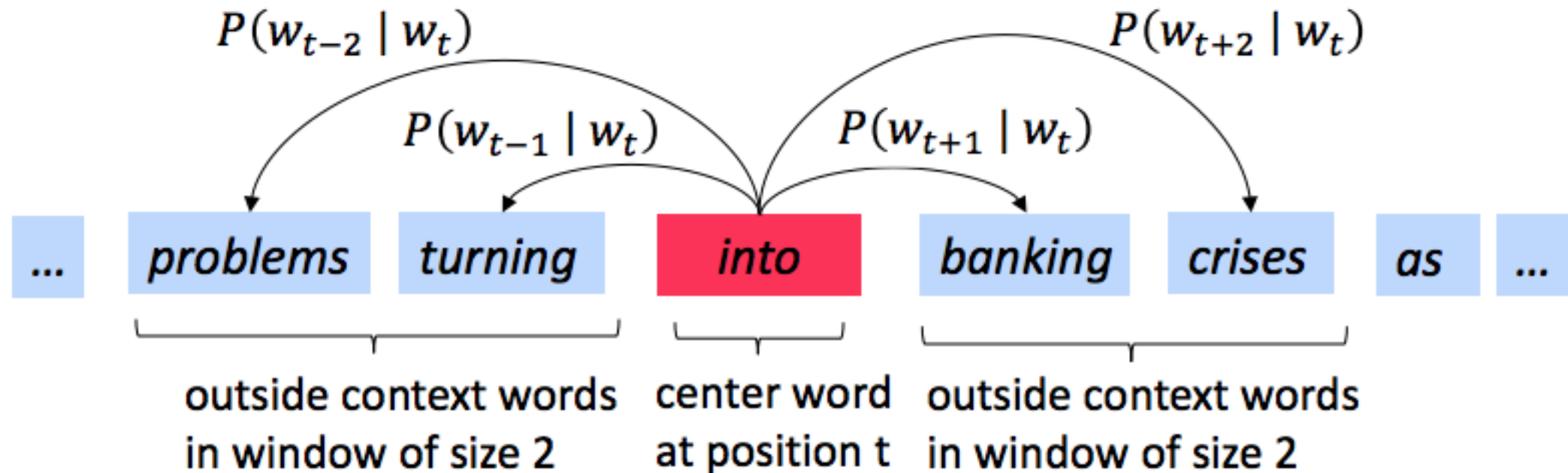
SVD Decomposition

Limitation of Co-occurrence Matrix

- The dimensions of the matrix change very often (new words are added very frequently and corpus changes in size).
- The matrix is extremely sparse since most words do not co-occur.
- The matrix is very high dimensional in general ($\approx 10^6 \times 10^6$)
- Quadratic cost to train (i.e. to perform SVD)
 - SVD based methods do not scale well for big matrices and it is hard to incorporate new words or documents. Computational cost for a $m \times n$ matrix is $O(mn^2)$
- Requires the incorporation of some hacks on X to account for the drastic imbalance in word frequency

Word2Vec – Skip-Gram Model

- Word2vec (Mikolov et al. 2013) is a framework for learning word vectors.



Semi-Supervised Scheme

Source Text

The quick brown fox jumps over the lazy dog. →

The quick brown fox jumps over the lazy dog. →

The quick brown fox jumps over the lazy dog. →

The quick brown fox jumps over the lazy dog. →

Training Samples

(the, quick)
(the, brown)

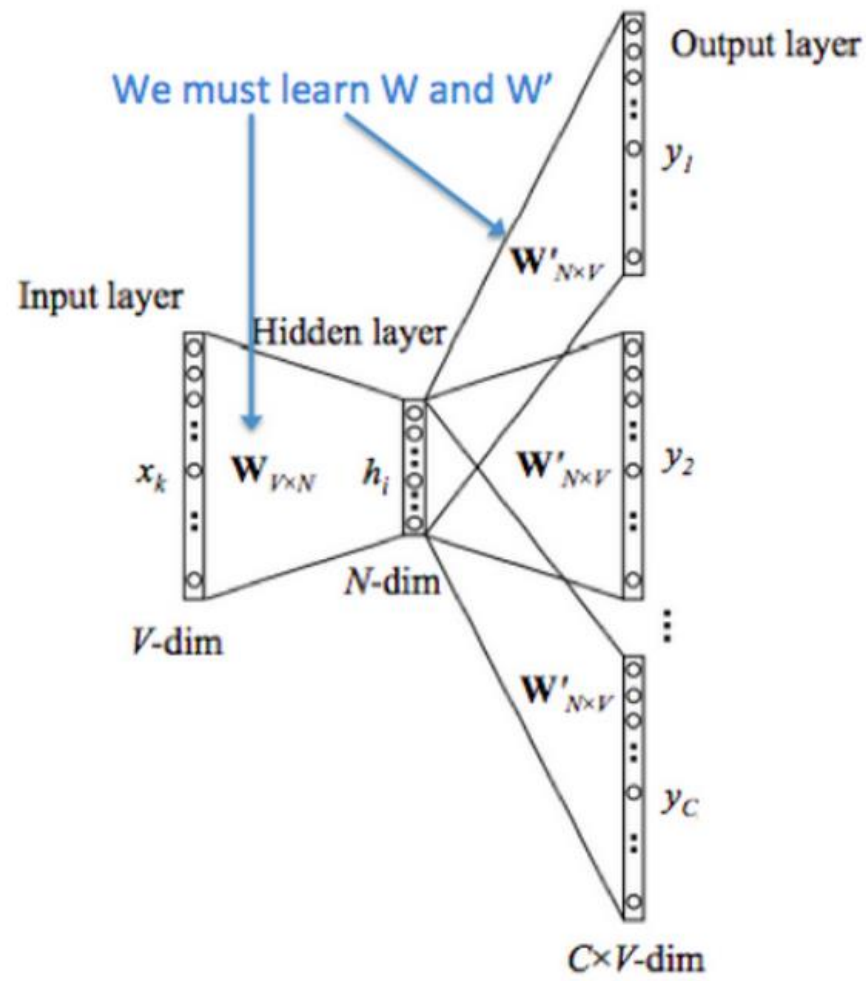
(quick, the)
(quick, brown)
(quick, fox)

(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)

(fox, quick)
(fox, brown)
(fox, jumps)
(fox, over)

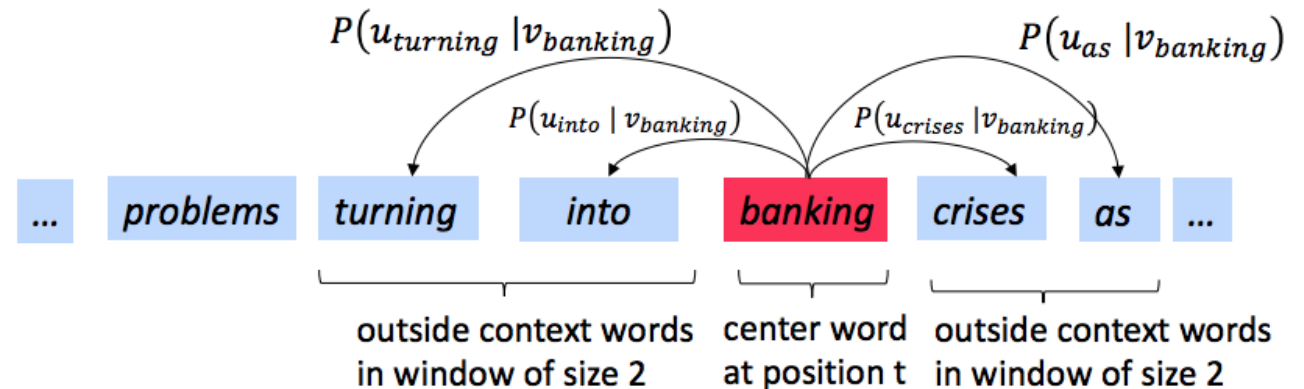
- We have a large corpus of text
- Every word in a fixed vocabulary is represented by a vector
- Go through each position t in the text, which has a centre word c and context (“outside”) words o
- Use the similarity of the word vectors for c and o to calculate the probability of o given c (or vice versa)
- Keep adjusting the word vectors to maximize this probability

Skip-Gram Model – Intuition



Notation for Skip-Gram Model:

- w_i : Word i from vocabulary V
- $\mathcal{V} \in \mathbb{R}^{n \times |V|}$: Input word matrix
- v_i : i -th column of $\mathcal{V}$, the input vector representation of word w_i
- $\mathcal{U} \in \mathbb{R}^{n \times |V|}$: Output word matrix
- u_i : i -th row of $\mathcal{U}$, the output vector representation of word w_i



Skip-Gram Model – Formulation

- For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given centre word w_j

Likelihood =

θ is all variables
to be optimized

$$L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

Strong Naïve Bayes Assumption

The objective function J is the (average) negative log likelihood.

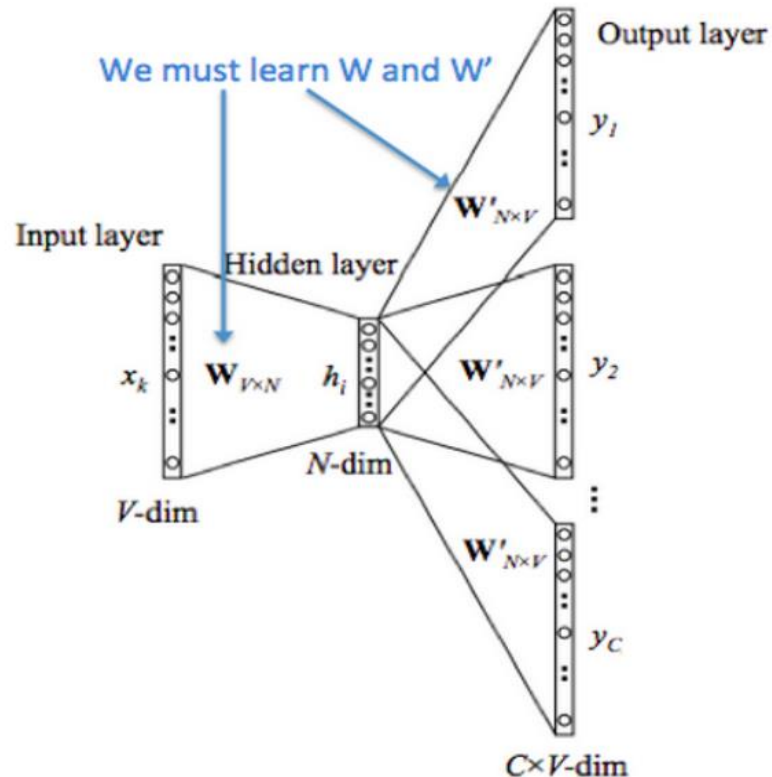
$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

Predictive Model

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

How to calculate $P(w_{t+j} | w_t; \theta)$?



$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

v_w when w is a center word

u_w when w is a context word

Dot product compares similarity of o and c .
Larger dot product = larger probability

After taking exponent,
normalize over entire vocabulary

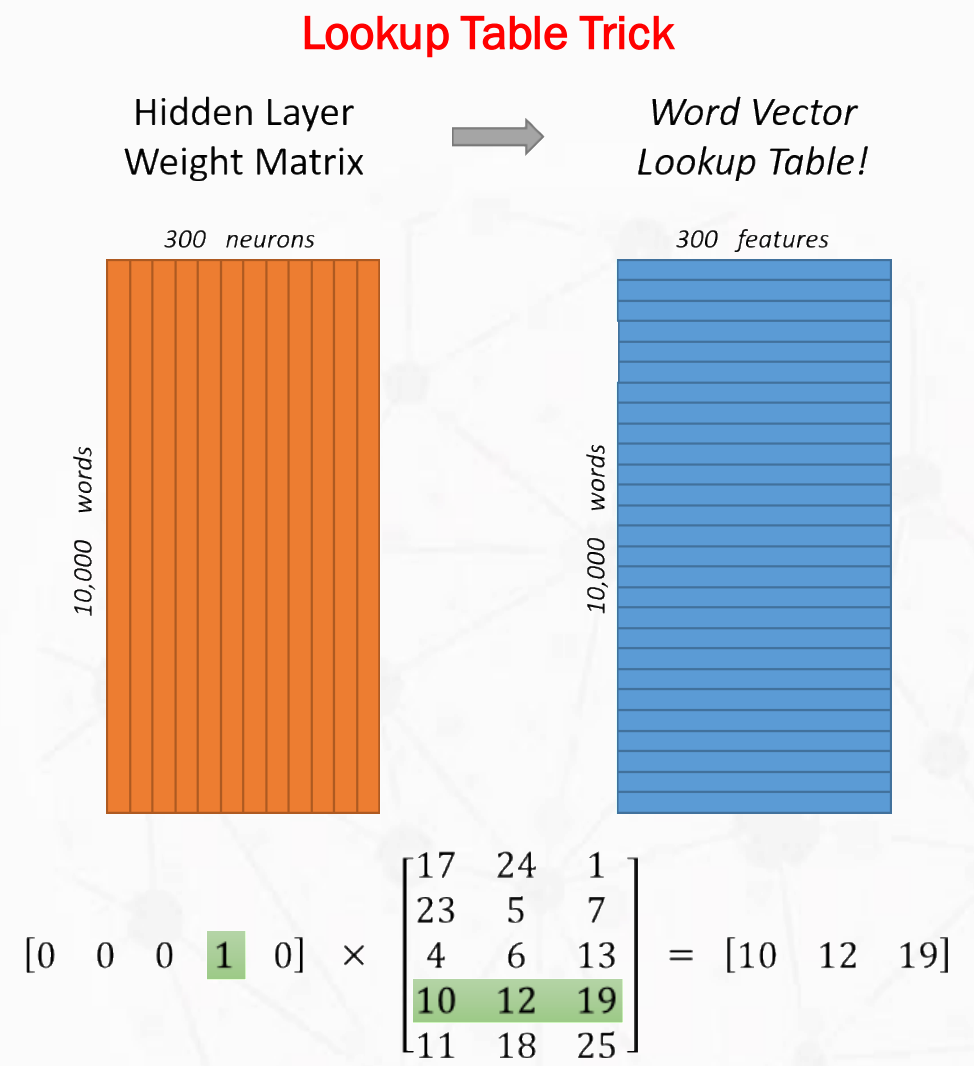
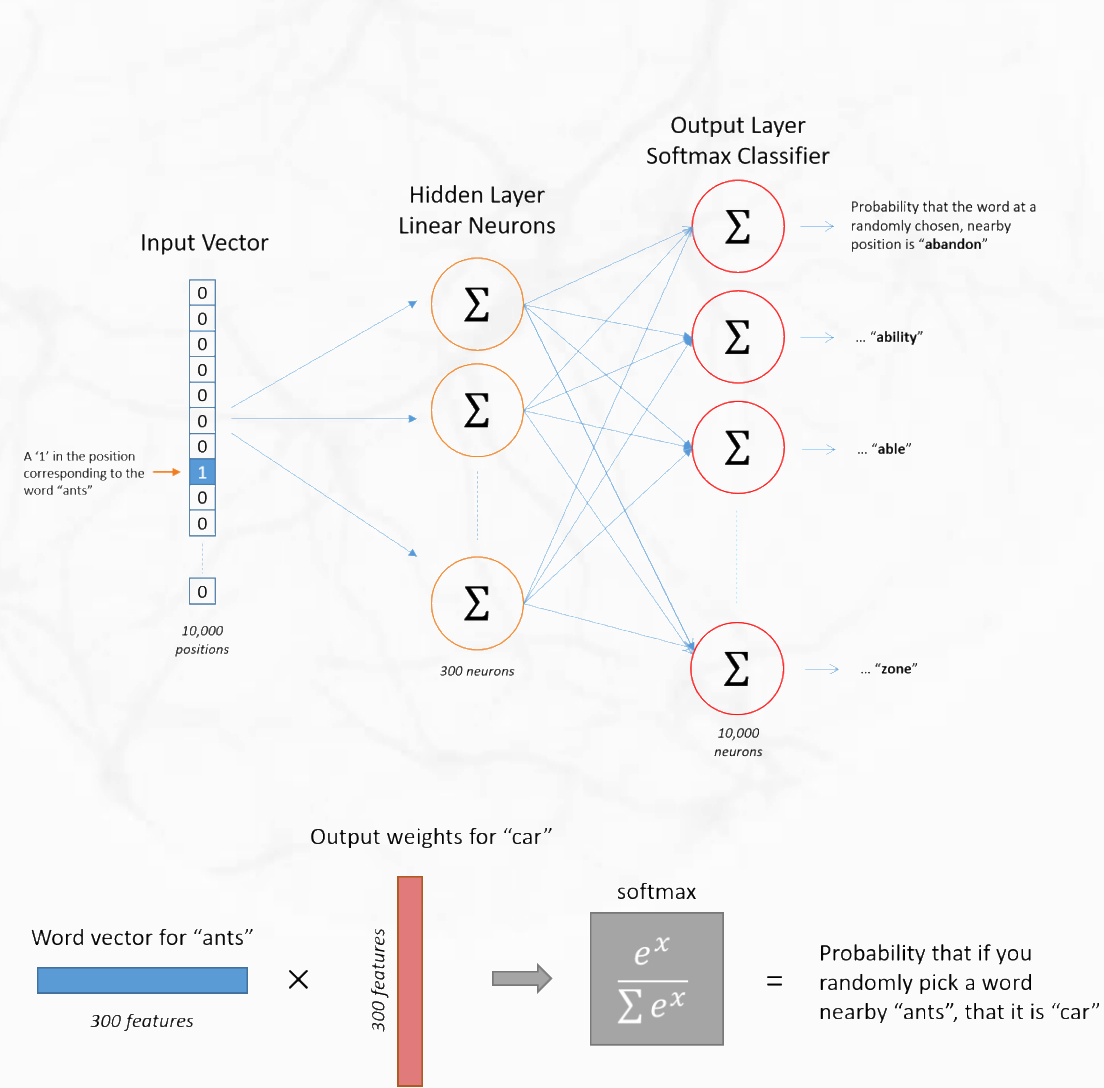
This is an example of the **softmax function** $\mathbb{R}^n \rightarrow \mathbb{R}^n$

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

The softmax function maps arbitrary values x_i to a probability distribution p_i

- “max” because amplifies probability of largest x_i
- “soft” because still assigns some probability to smaller x_i
- Frequently used in Deep Learning

Implementation



The scatter plot displays the distribution of 100 words from the Brown Corpus, plotted against the first two principal components (PC1 and PC2). The x-axis (PC1) ranges from approximately -12 to 8, and the y-axis (PC2) ranges from approximately -2 to 6. The words are scattered across the plot, with some words appearing in clusters and others isolated. The plot shows a clear separation between words that are more frequent or more central in the corpus and those that are more peripheral.

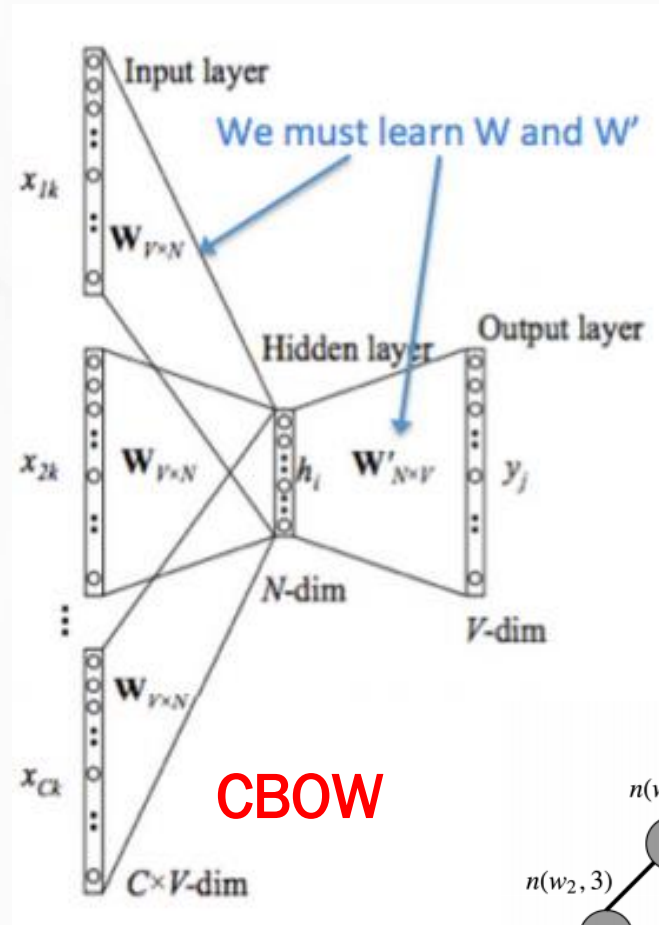
```

Average loss at step 92000 : 3.485070344209671
Average loss at step 94000 : 3.4761118290424347
Average loss at step 96000 : 3.5120878417491914
Average loss at step 98000 : 3.408551989376545
Average loss at step 100000 : 3.459366007089615
Nearest to over: linker, chap, discounted, faq, unreleased, stadtbahn, rj, jamaat,
Nearest to war: opportunity, reductionism, spectra, dollfuss, eightfold, marque, propto, episode,
Nearest to this: it, which, the, that, some, any, guthrie, there,
Nearest to years: months, swells, definitions, vastly, dhimma, scaling, webelements, days,
Nearest to often: sometimes, still, commonly, usually, generally, widely, also, enlisted,
Nearest to three: five, four, seven, two, six, eight, nine, zero,
Nearest to he: it, she, they, there, who, later, largely, urging,
Nearest to states: kingdom, literature, nilsson, contractor, fanzines, disquiet, ordained, pilot,
Nearest to may: can, would, will, must, cannot, could, might, should,
Nearest to time: stories, ipv, play, way, fronts, axelrod, milieu, money,
Nearest to system: discretionary, episcopacy, systems, economy, monthly, firearms, kids, concerns,
Nearest to have: had, has, were, are, be, pendleton, defy, onstage,
Nearest to many: some, several, these, various, all, other, lrv, such,
Nearest to most: more, respectfully, malted, many, regulars, withholding, appropriated, bonewits,
Nearest to more: less, postcard, gasoline, most, affects, very, bravely, exceedingly,
Nearest to be: been, is, have, being, were, was, appear, are,
=====

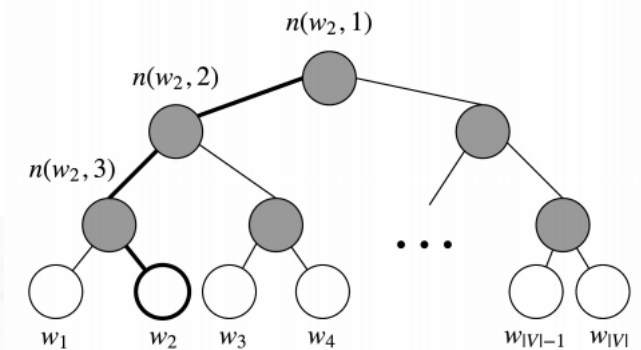
```

Word2Vec – Variants

- Two model variants:
 - Skip-grams (SG) Predict context ("outside") words (position independent) given centre word
 - Continuous Bag of Words (CBOW) Predict centre word from (bag of) context words
- Additional efficiency in training
 - Negative Sampling
 - Hierarchical softmax



Negative Sampling: Given that the output layer of our model has a weight matrix that's 300 x 10,000. So we will just be updating the weights for our positive word ("quick"), plus the weights for 5 other words that we want to output 0. That's a total of 6 output neurons, and 1,800 weight values total. That's only 0.06% of the 3M weights in the output layer!



Attention is All You Need

Fundamental Neural Architecture for LLMs

Spatial Attention

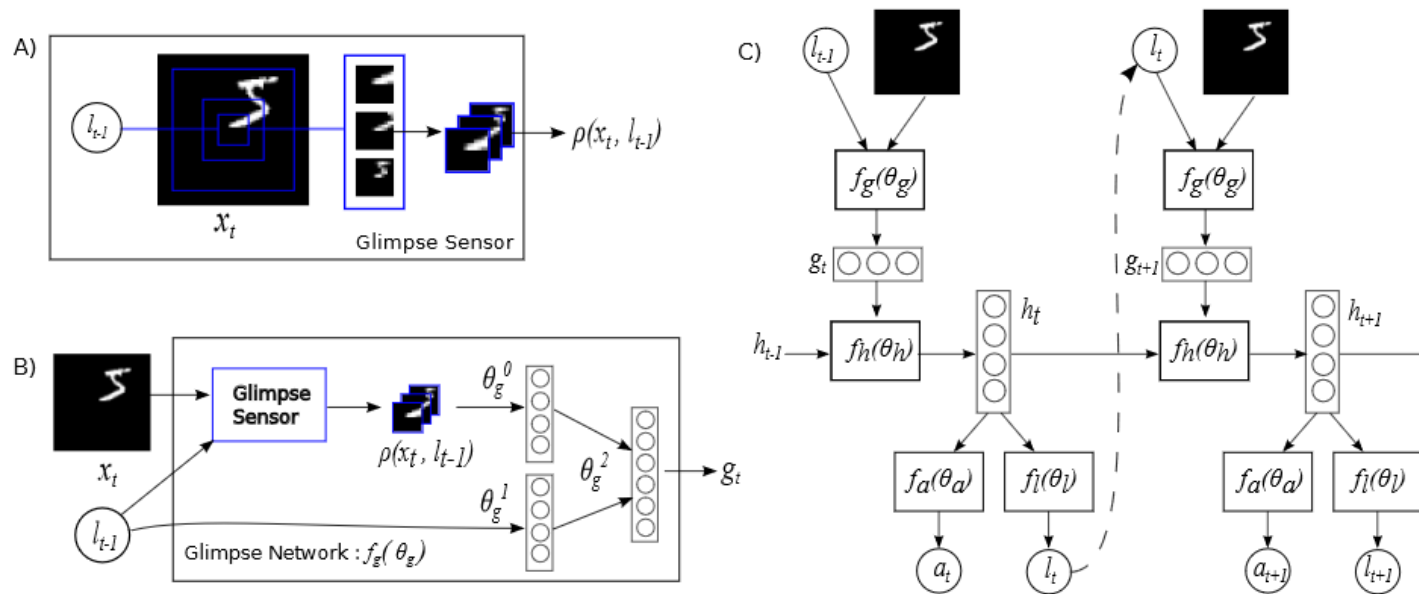
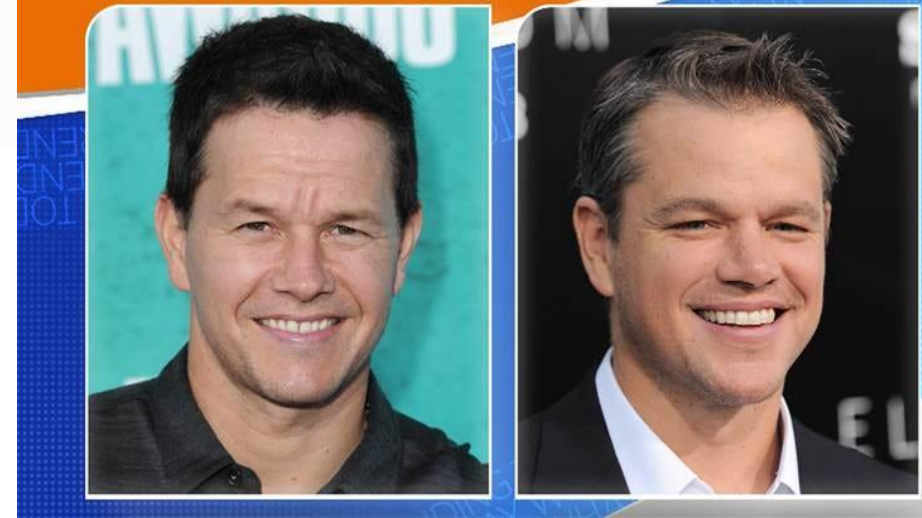
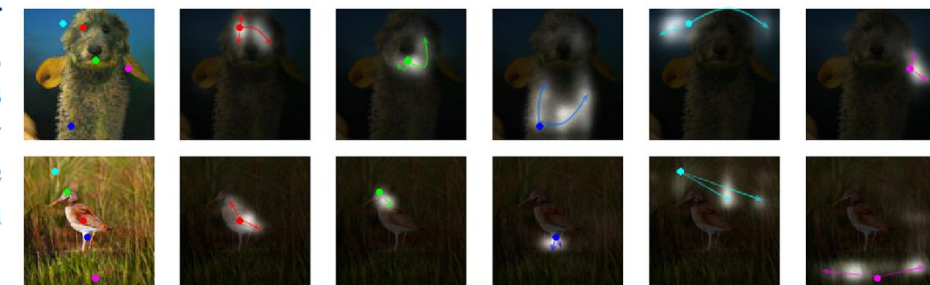


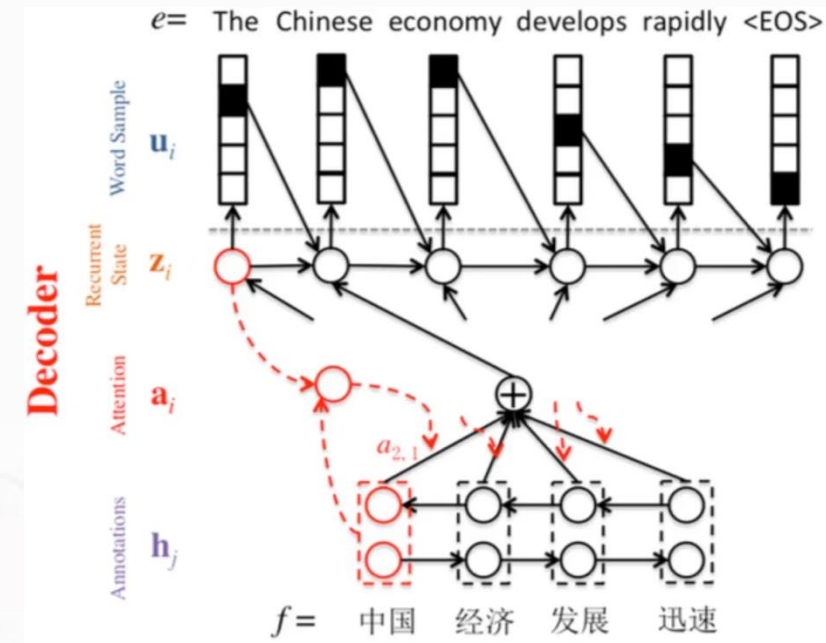
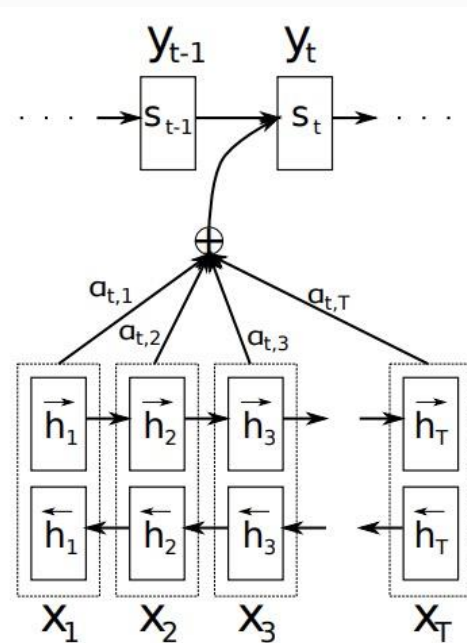
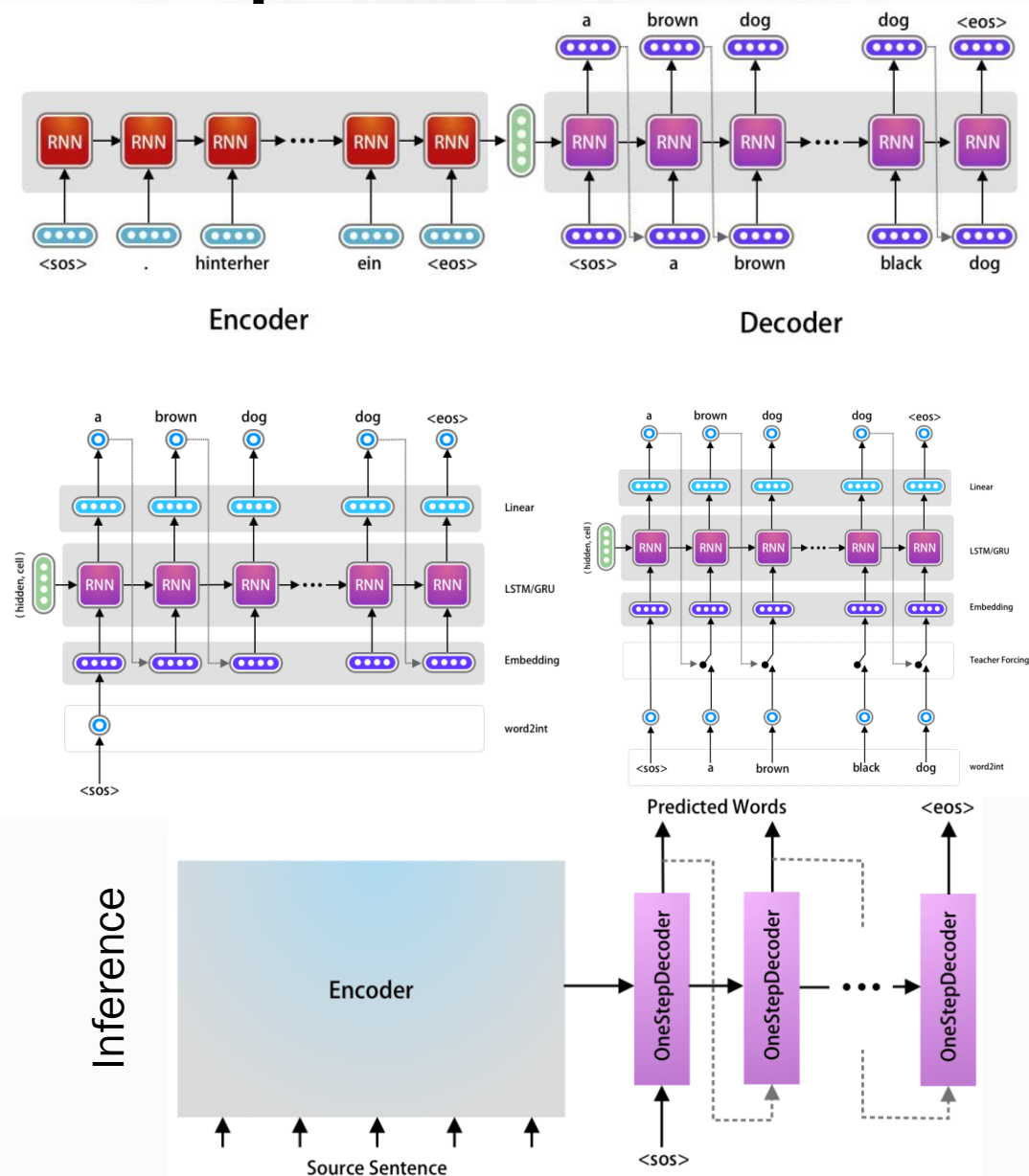
Figure 1: **A) Glimpse Sensor:** Given the coordinates of the glimpse and an input image, the sensor extracts a *retina-like* representation $\rho(x_t, l_{t-1})$ centered at l_{t-1} that contains multiple resolution patches. **B) Glimpse Network:** Given the location (l_{t-1}) and input image (x_t), uses the glimpse sensor to extract retina representation $\rho(x_t, l_{t-1})$. The retina representation and glimpse location is then mapped into a hidden space using independent linear layers parameterized by θ_g^0 and θ_g^1 respectively using rectified units followed by another linear layer θ_g^2 to combine the information from both components. The glimpse network $f_g(\cdot; \{\theta_g^0, \theta_g^1, \theta_g^2\})$ defines a trainable bandwidth limited sensor for the attention network producing the glimpse representation g_t . **C) Model Architecture:** Overall, the model is an RNN. The core network of the model $f_h(\cdot; \theta_h)$ takes the glimpse representation g_t as input and combining with the internal representation at previous time step h_{t-1} , produces the new internal state of the model h_t . The location network $f_l(\cdot; \theta_l)$ and the action network $f_a(\cdot; \theta_a)$ use the internal state h_t of the model to produce the next location to attend to l_t and the action/classification a_t respectively. This basic RNN iteration is repeated for a variable number of steps.



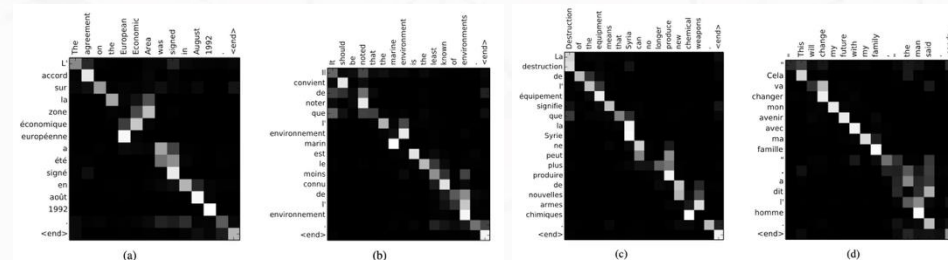
- Partially Observable Markov Decision Process
- Glimpse Sensor
 - Retina-Like Representation
- Actions
 - Location of Next Glimpse Sensor
 - Environment Action, i.e., Object Classification
- Information Integration



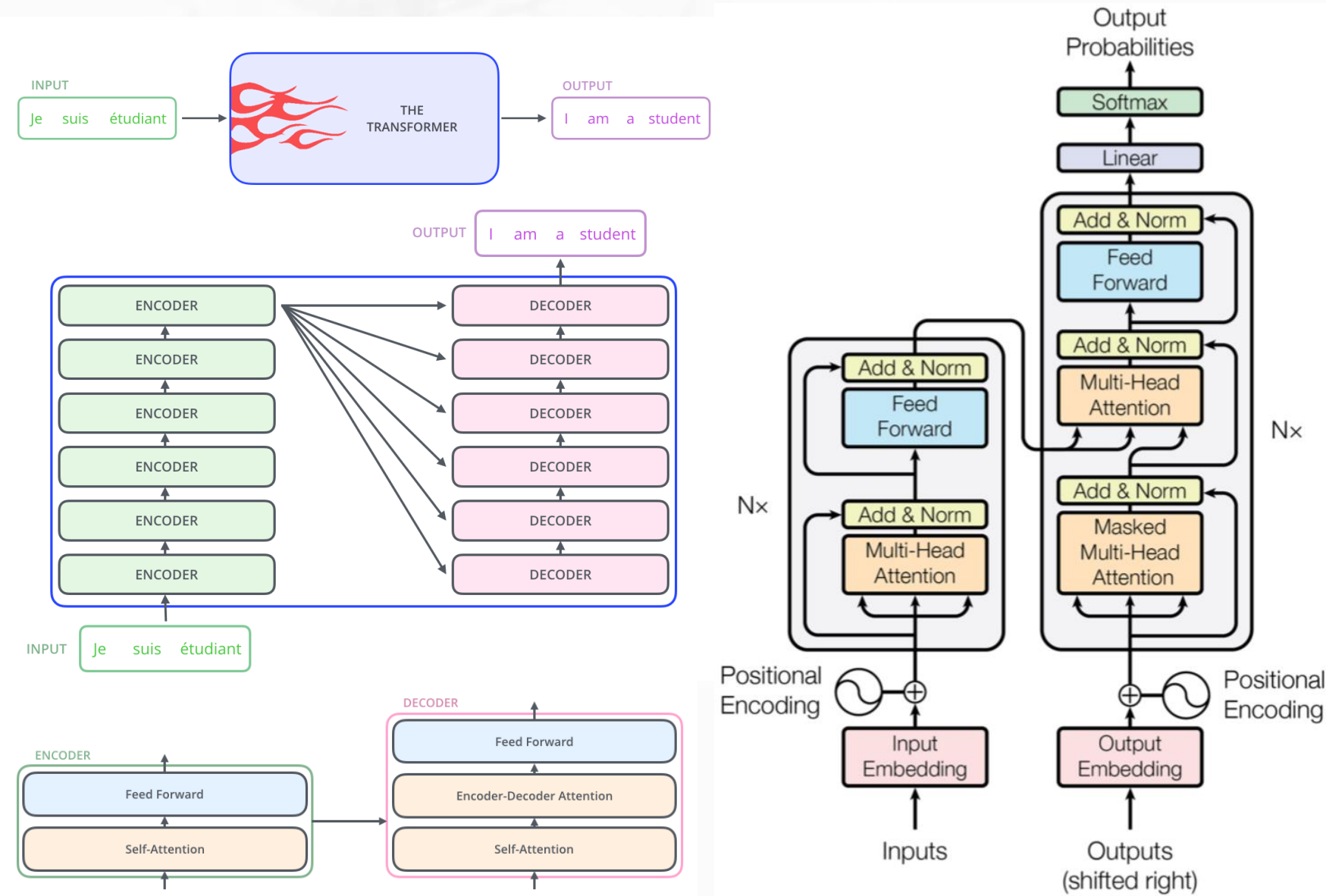
Temporal Attention



- Encoder-Decoder Framework
- Encoding with Bi-Directional RNNs $h_j = [\vec{h}_j; \overleftarrow{h}_j]$ $s_i = f(s_{i-1}, y_{i-1}, c_i)$
- Decoding with Additional Context Vector $p(y_i | y_1, y_2, \dots, y_{i-1}) = g(y_{i-1}, s_i, c_i)$
- Context is from All Inputs $c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$ $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$
- Attention Aligns with Outputs $e_{ij} = a(s_{i-1}, h_j)$
- Training Trick: Teacher Forcing for Fast Convergence

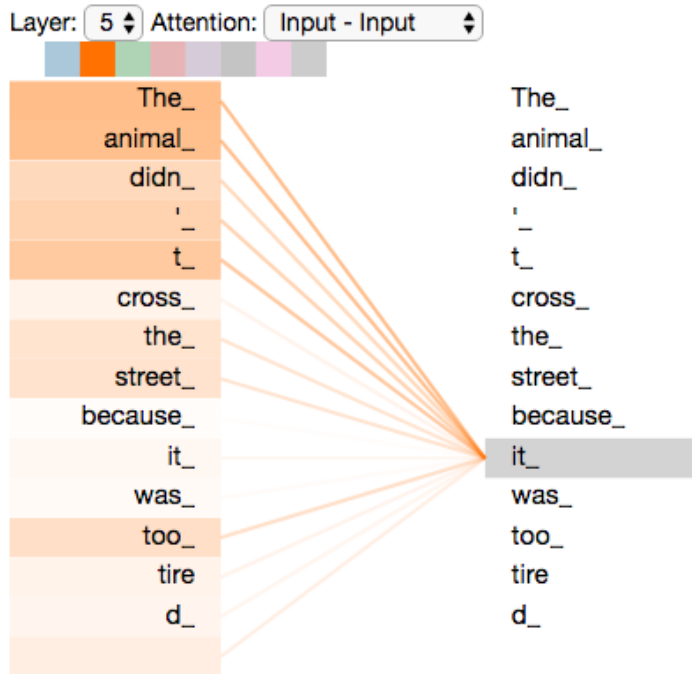
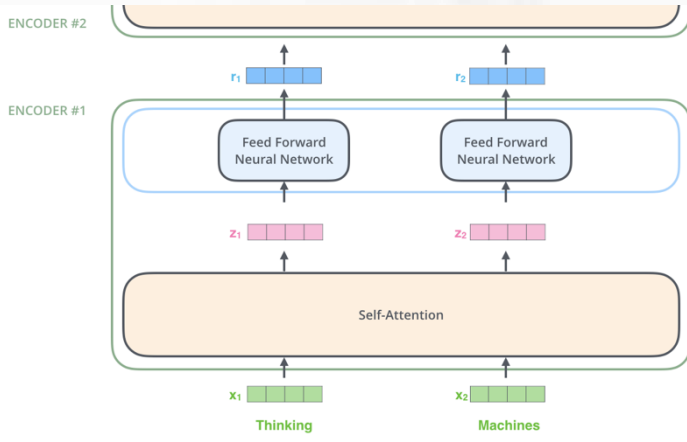


Attention Is All You Need:



Name	Alignment score function
Content-base attention	$\text{score}(s_t, h_i) = \text{cosine}[s_t, h_i]$
Additive(*)	$\text{score}(s_t, h_i) = \mathbf{v}_a^T \tanh(\mathbf{W}_a [s_t; h_i])$
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a s_t)$ Note: This simplifies the softmax alignment to only depend on the target position.
General	$\text{score}(s_t, h_i) = s_t^T \mathbf{W}_a h_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.
Dot-Product	$\text{score}(s_t, h_i) = s_t^T h_i$
Scaled Dot-Product(^)	$\text{score}(s_t, h_i) = \frac{s_t^T h_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.

Transformer (2) : Self-Attention



Input

Embedding

Queries

Keys

Values

Score

Divide by $8 (\sqrt{d_k})$

Softmax

Softmax

X
Value

Sum

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

v_1

z_1

Machines

x_2

q_2

k_2

v_2

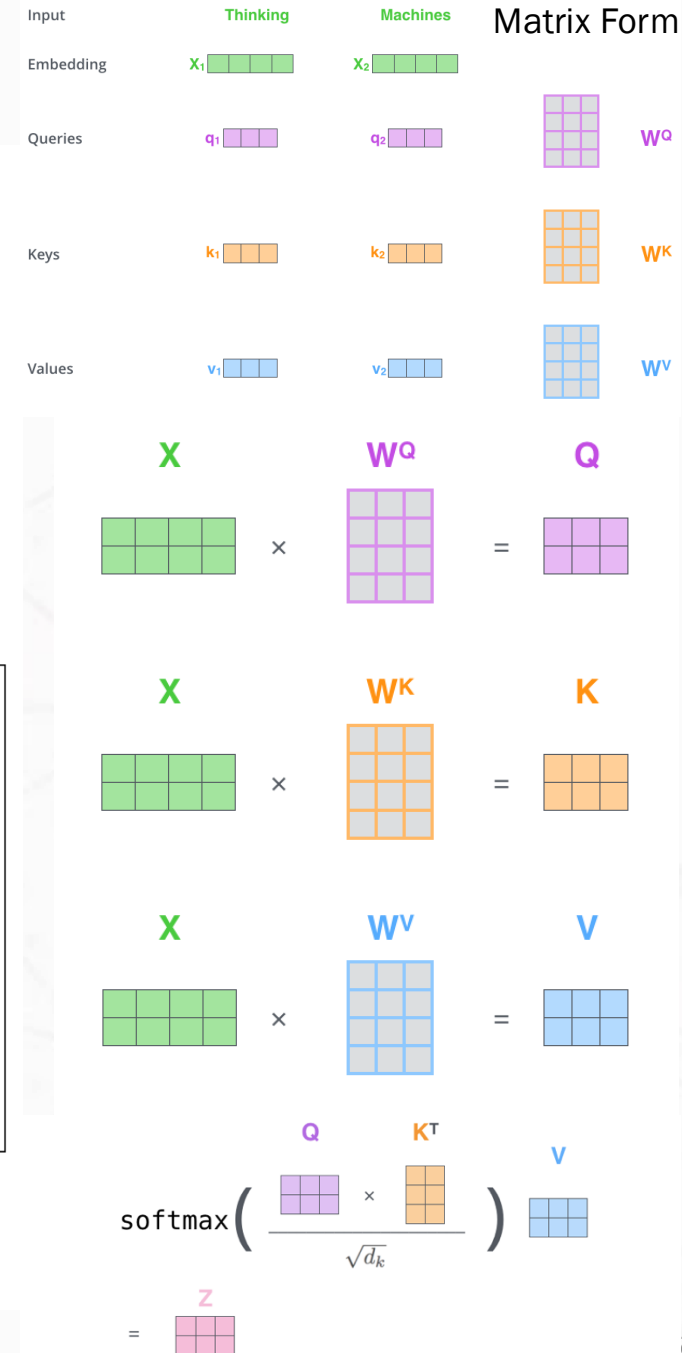
$q_2 \cdot k_2 = 96$

12

0.12

v_2

z_2



Transformer (3) : Multi-Head Attention

1) This is our input sentence*

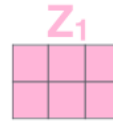
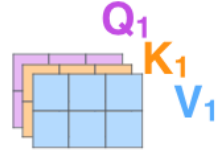
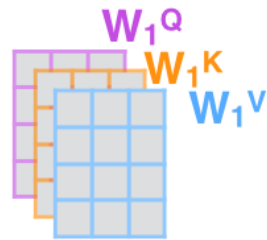
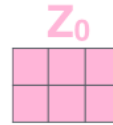
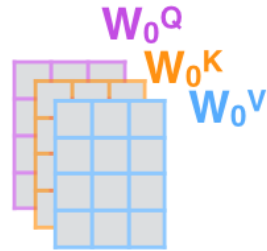
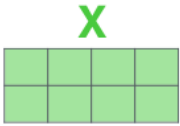
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

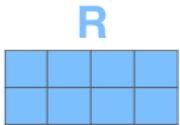
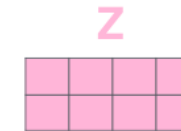
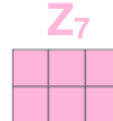
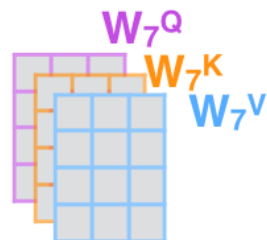
Thinking Machines



...

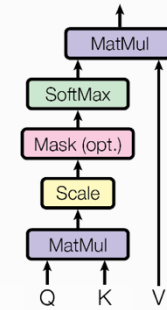
...

...

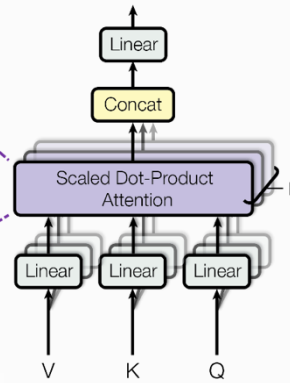


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

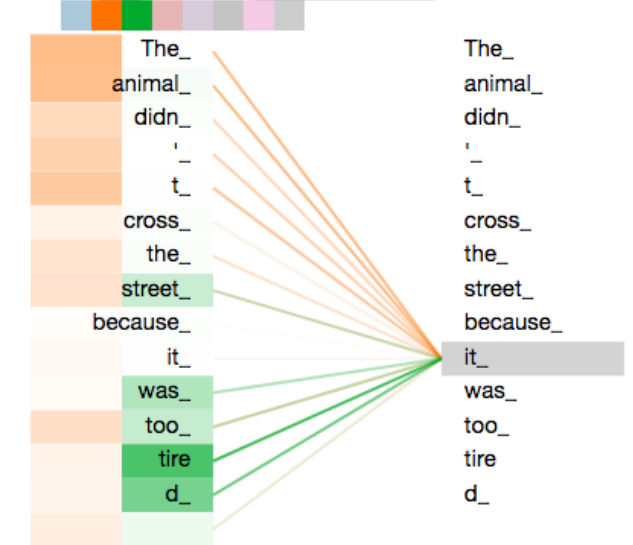
Scaled Dot-Product Attention



Multi-Head Attention



Layer: 5 Attention: Input - Input



- It expands the model's ability to focus on different positions.
- It gives the attention layer multiple "representation subspaces".

The diagram illustrates the Transformer architecture for machine translation, showing the flow from input to output through multiple layers of encoders and decoders.

High-level Overview:

- ENCODER #0** and **ENCODER #1** process the input sequence (Je suis étudiant) into embeddings.
- DECODER #0** and **DECODER #1** generate the output sequence (I am a student).

Detailed Encoder/Decoder Structure:

- ENCODER #1** (Left): Consists of multiple layers. Each layer includes:
 - Self-Attention** (orange box)
 - Add & Normalize** (green box)
 - Feed Forward** (blue box)
 - Add & Normalize** (green box)
- DECODER #1** (Right): Consists of multiple layers. Each layer includes:
 - Encoder-Decoder Attention** (orange box)
 - Add & Normalize** (green box)
 - Self-Attention** (orange box)
 - Add & Normalize** (green box)
 - Feed Forward** (blue box)
 - Add & Normalize** (green box)

Positional Encoding:

- EMBEDDING WITH TIME SIGNAL**: Shows the input sequence (Je suis étudiant) and the corresponding positional encodings (t1, t2, t3).
- POSITIONAL ENCODING**: Shows the positional encodings (t1, t2, t3) and the corresponding embeddings (x1, x2, x3).
- EMBEDDINGS**: Shows the input sequence (Je suis étudiant) and the corresponding embeddings (x1, x2, x3).
- INPUT**: Shows the input sequence (Je suis étudiant) and the corresponding embeddings (x1, x2, x3).

Positional Encoding Formulas:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Positional Encoding Visualizations:

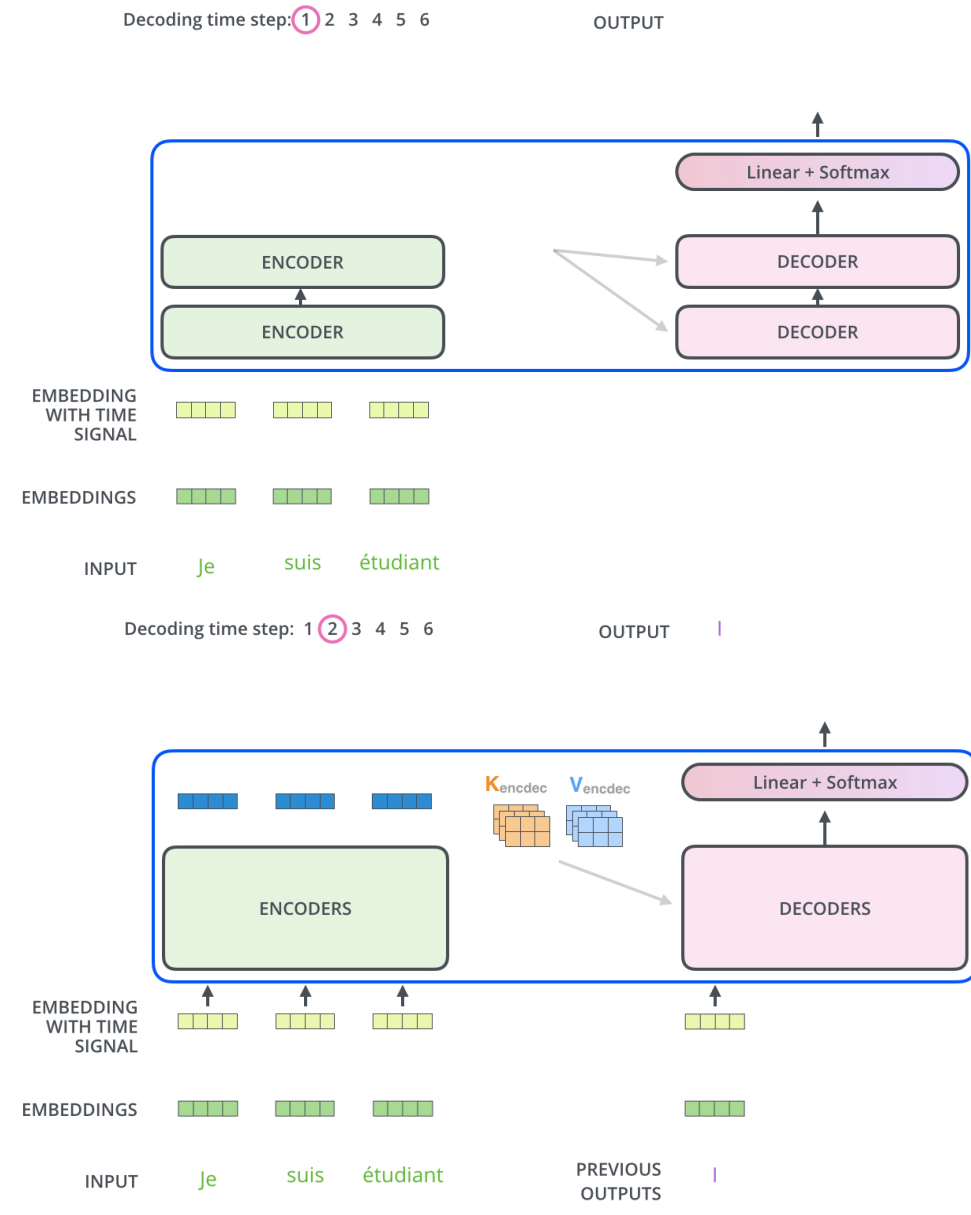
- (a) Sinusoidal**: Heatmap showing the sinusoidal pattern of positional encodings.
- (b) Position embedding**: Heatmap showing the learned position embeddings.
- (c) FLOATER**: Heatmap showing the learned positional encodings.
- (d) RNN**: Heatmap showing the learned positional encodings.

Legend:

- Thinking**: Thinking
- Machines**: Machines

1. Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).
2. <https://jalammar.github.io/illustrated-transformer/>

Transformer (5) : Decoding & Loss



Which word in our vocabulary is associated with this index?

Get the index of the cell with the highest value (argmax)

log_probs



5

am

logits



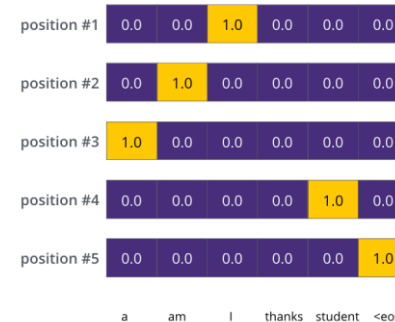
Softmax

Linear

Decoder stack output

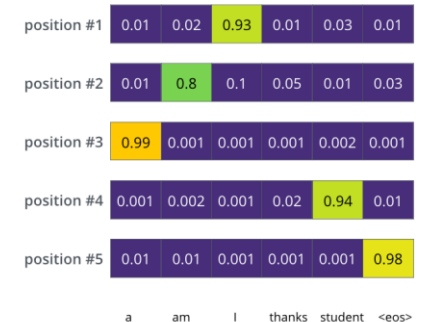
Target Model Outputs

Output Vocabulary: a am l thanks student <eos>



Trained Model Outputs

Output Vocabulary: a am l thanks student <eos>



Causal Language Modelling

GPT - Scaling-Up Decoder-Only Model for Generation

Text Generation Task

[–] The issue focused on the way that the city's police officers have reacted in recent years to the deaths of Michael Brown in Ferguson, Mo., Eric Garner in New York City and Sandra Bland in Texas, as well as the shooting of unarmed teen Michael Brown by a white police officer in Ferguson, Mo. ...

[Military] The issue focused on the fact that the **government** had spent billions on the **military** and that it could not **deploy** the **troops** in time. The prime minister said that the **country** would take back **control** of its **airspace** over Syria in the next 48 hours. \n The **military** is investigating why...

[Space] The issue focused on a series of incidents that occurred in the past few months, which included an alleged attack by Islamic State fighters on a Kurdish checkpoint, the use of **drones** in combat, **space** technology research by Russian and American **space** companies, and more. \n The **world**...

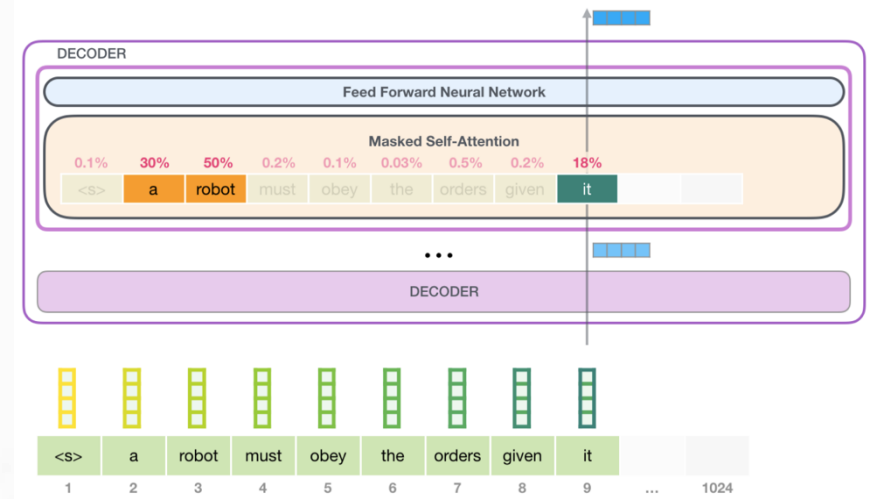
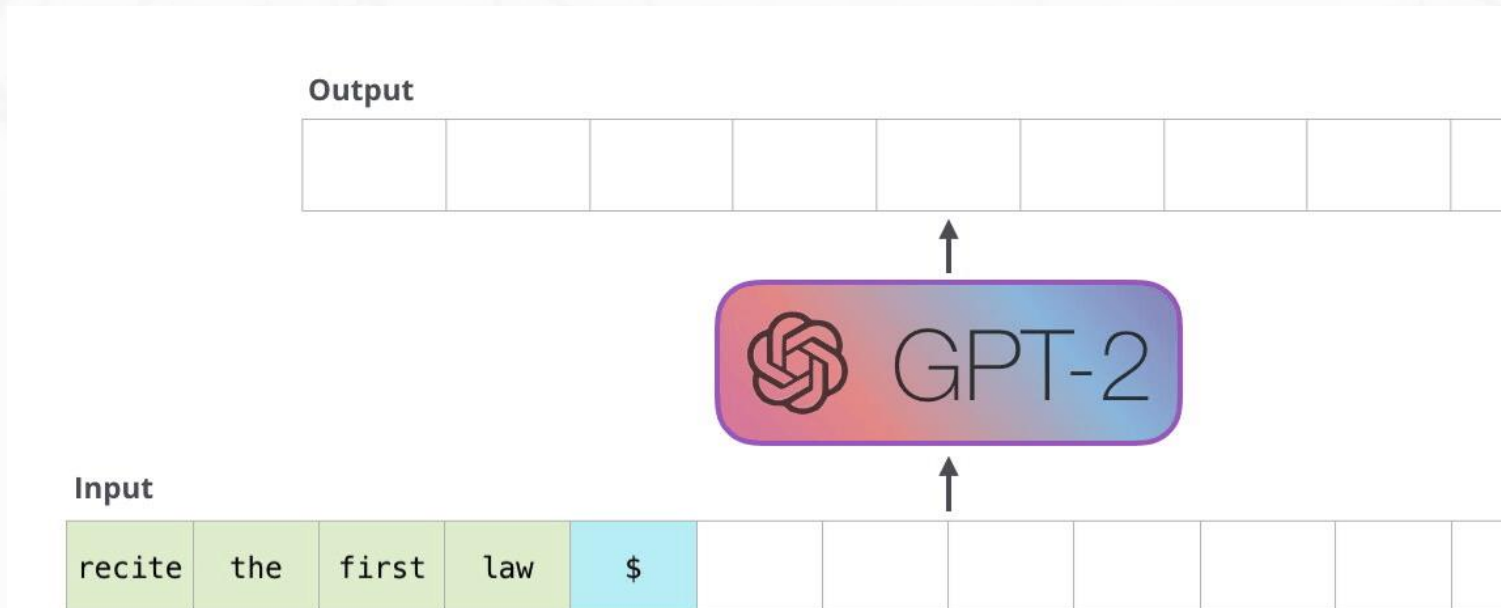
[Science] The issue focused on a single piece: the **question** "What is the meaning of life?" This **question** has puzzled many **philosophers**, who have attempted to **solve** it by using some of the **concepts** of **quantum mechanics**, but they have to **solve** it by the **laws** of **nature** themselves...

[Politics] The issue focused on a single section of the **legislation**. It's unclear whether the **committee** will **vote** to extend the **law**, but the **debate** could have wider implications. \n "The issue of the **law's** applicability to the **United Kingdom's referendum campaign** has been one of...

[Computers] The issue focused on the role of social **media** as a **catalyst** for political and corporate engagement in the **digital** economy, with the aim of encouraging companies to use the power of social **media** and the **Internet** to reach out to their target market. \n ...

Auto-Regressive Generative Model

- For example, in the example on the right, the token “it” attends to “a robot” and “it” in the previous token sequence on the left, since the right sequence is unobserved and to be generated.
- The output of the masked self-attention layer is the weighted average of the values of the tokens with the attention weights learned (30% for “a”, 50% for “robot” and 18% for “it”).
- we first calculate the attention scores through a product of queries and keys, as the same in self-attention. Then we apply an attention mask to make the upper right matrix a negative infinity.



Queries X **Keys** = **Scores (before softmax)**

robot	must	obey	orders	0.11	0.00	0.81	0.79
robot	must	obey	orders	0.19	0.50	0.30	0.48
robot	must	obey	orders	0.53	0.98	0.95	0.14
robot	must	obey	orders	0.81	0.86	0.38	0.90

Scores (before softmax) → **Apply attention mask** → **Masked Scores (before softmax)** → **Softmax along row** → **Scores**

0.11	0.00	0.81	0.79	0.11	-inf	-inf	-inf	1	0	0	0
0.19	0.50	0.30	0.48	0.19	0.50	-inf	-inf	0.48	0.52	0	0
0.53	0.98	0.95	0.14	0.53	0.98	0.95	-inf	0.31	0.35	0.34	0
0.81	0.86	0.38	0.90	0.81	0.86	0.38	0.90	0.25	0.26	0.23	0.26

Training Dataset

Article #1 tokens	<summarize>	Article #1 Summary
Article #2 tokens	<summarize>	Article #2 Summary
Article #3 tokens	<summarize>	Article #3 Summary

Training Dataset

I	am	a	student	<to-fr>	je	suis	étudiant
let	them	eat	cake	<to-fr>	Qu'ils	mangent	de
good	morning	<to-fr>	Bonjour				



GPT - Generative Pre-Training Transformer & Finetuning

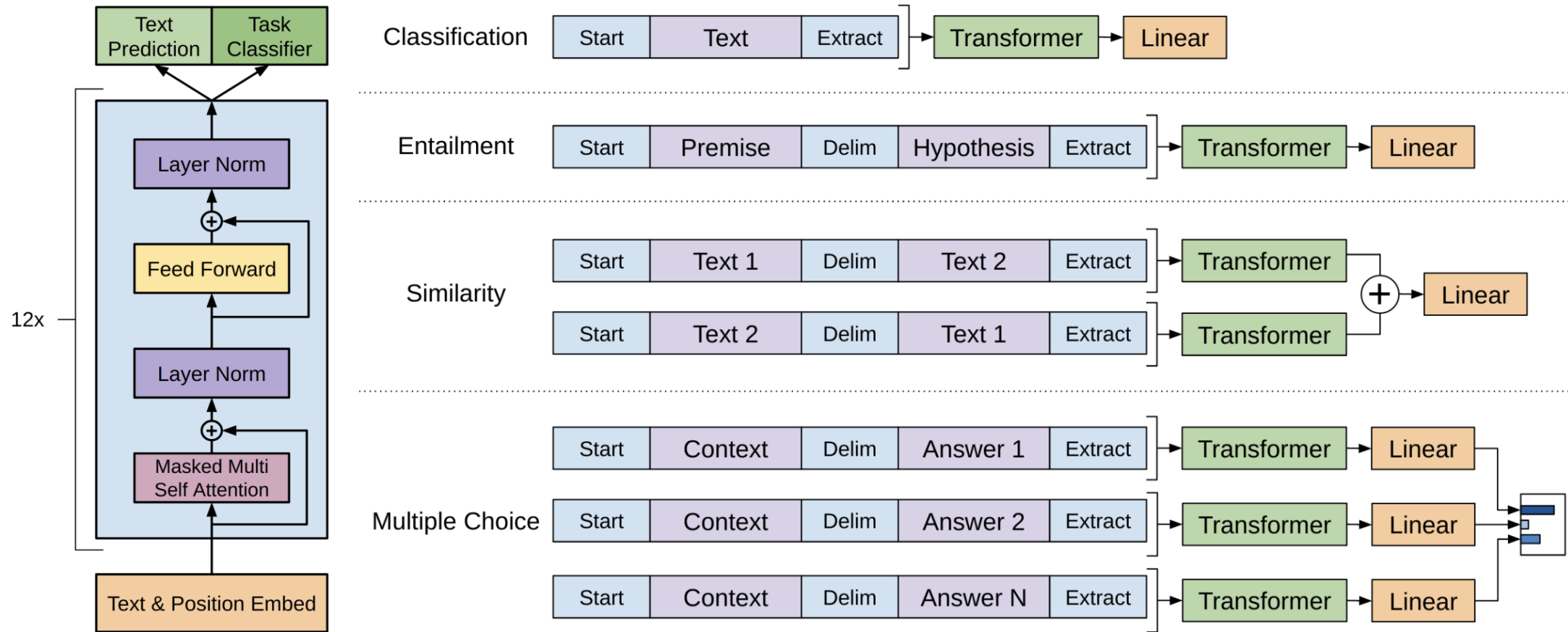
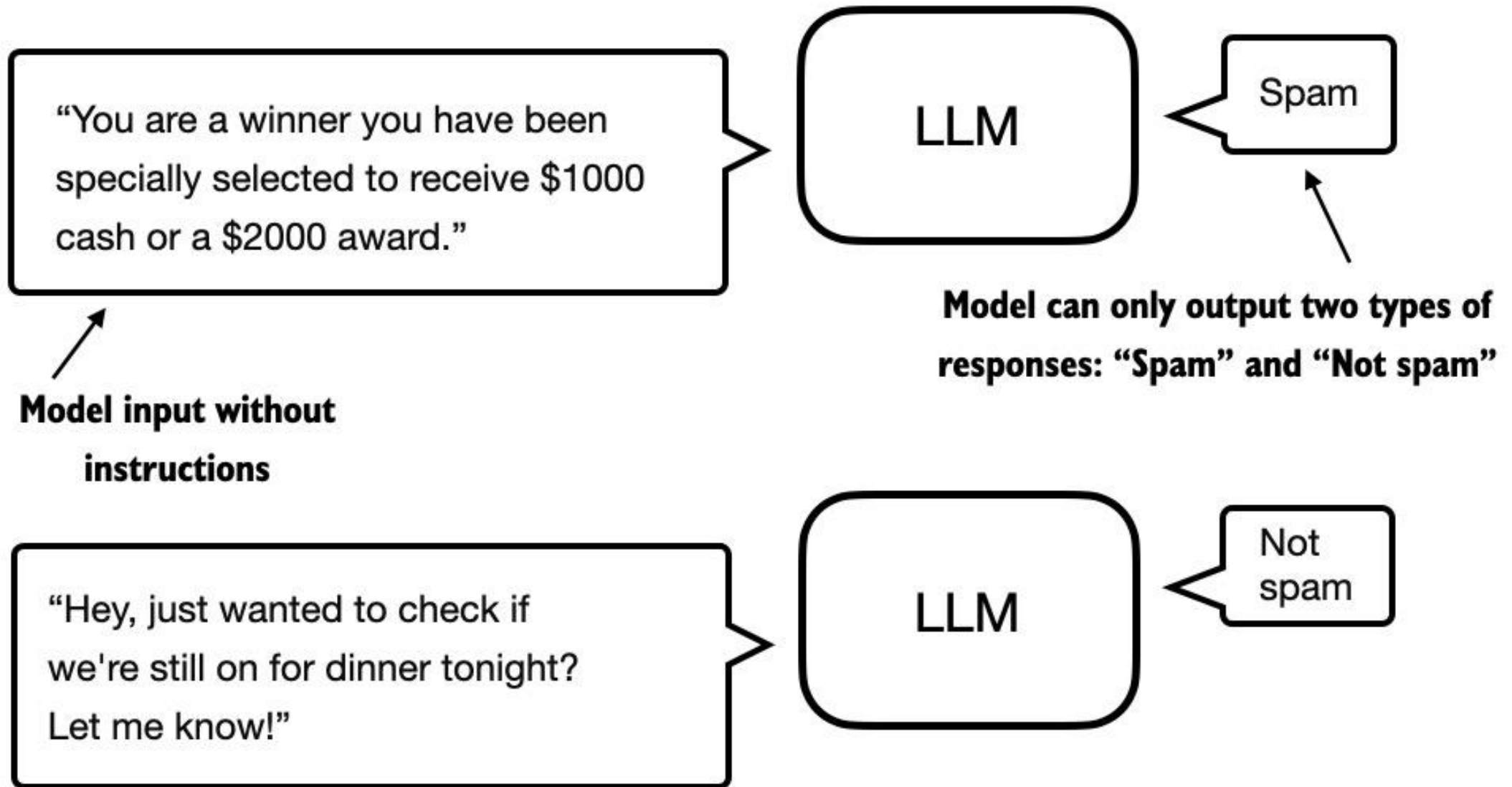


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

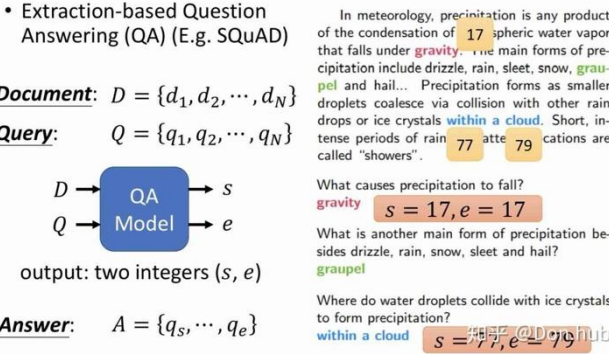
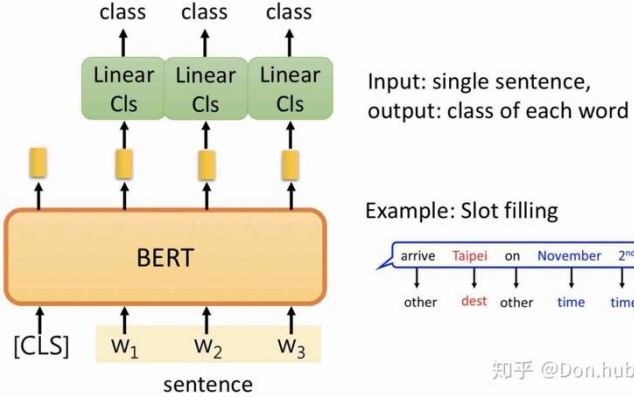
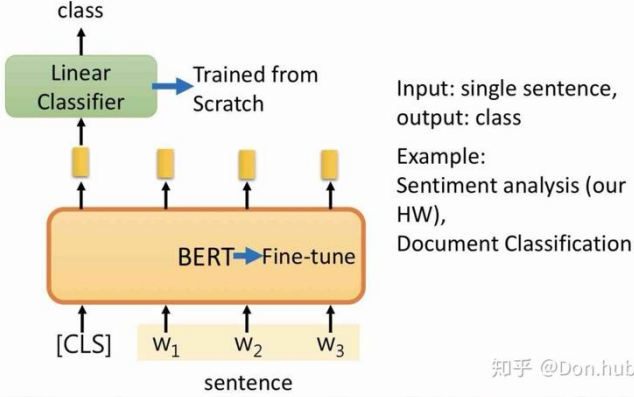
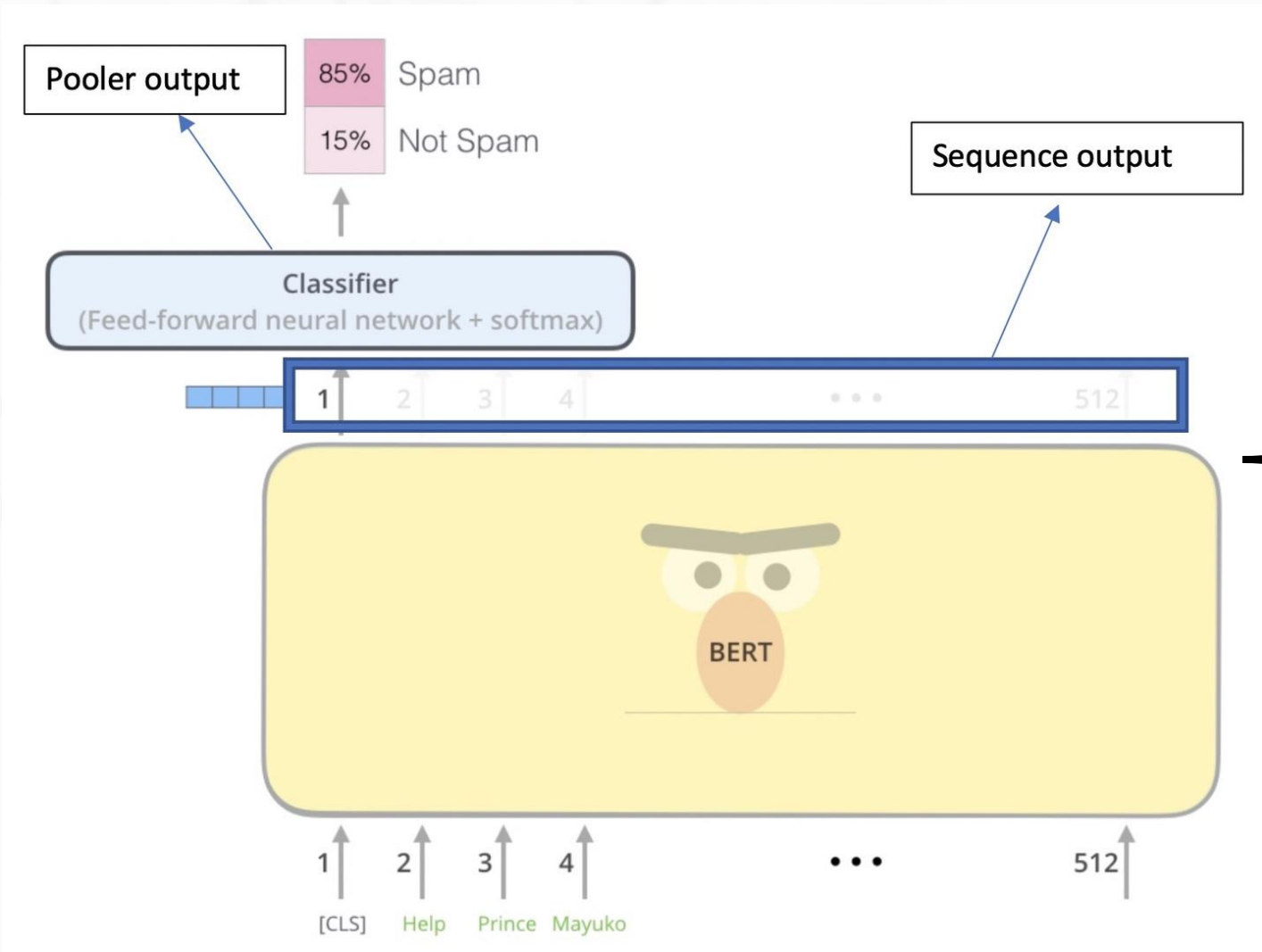
Masked Language Modelling

BERT - Scaling-Up Encoder-Only Model for Understanding

Text Classification Task

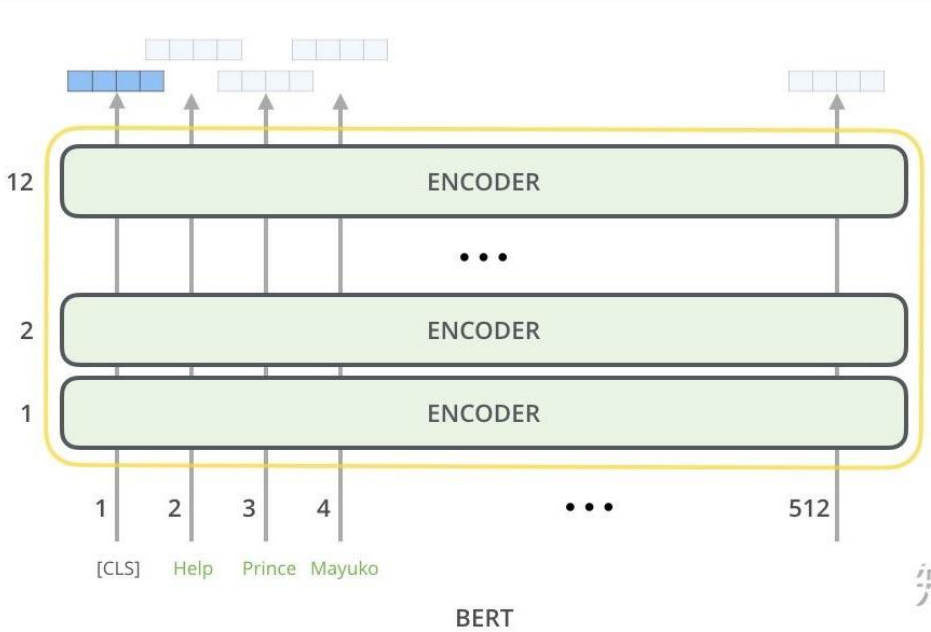
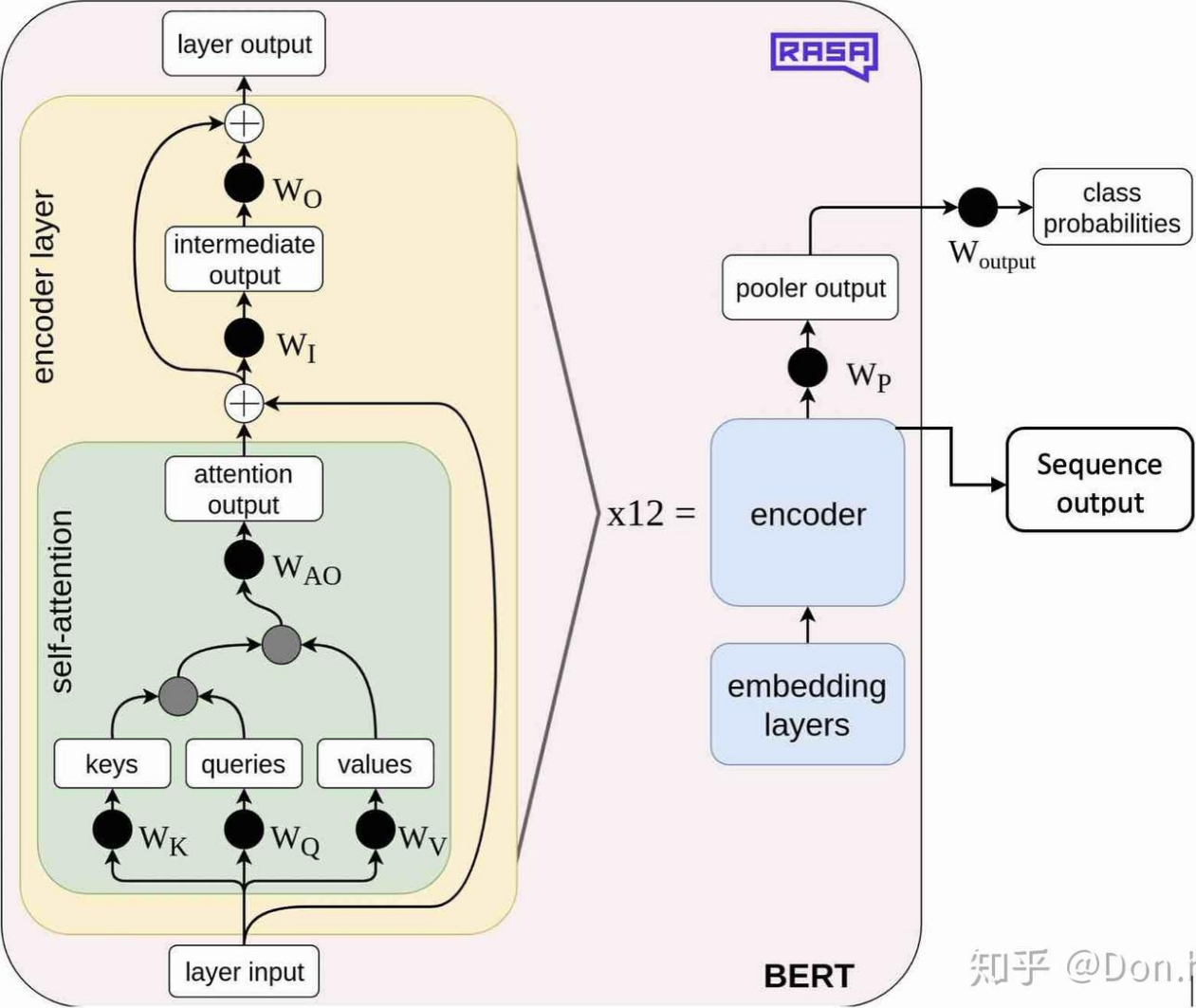


Bidirectional Encoder Representations from Transformers



1. Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2019, June. Bert: Pretraining of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (pp. 4171-4186).

Encoder-Only Model



Stacking Multiple Encoder

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	#ing	[SEP]
Token Embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{\#ing}$	$E_{[SEP]}$
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

Input Embedding

知乎 @Don.hub

Training BERT Model

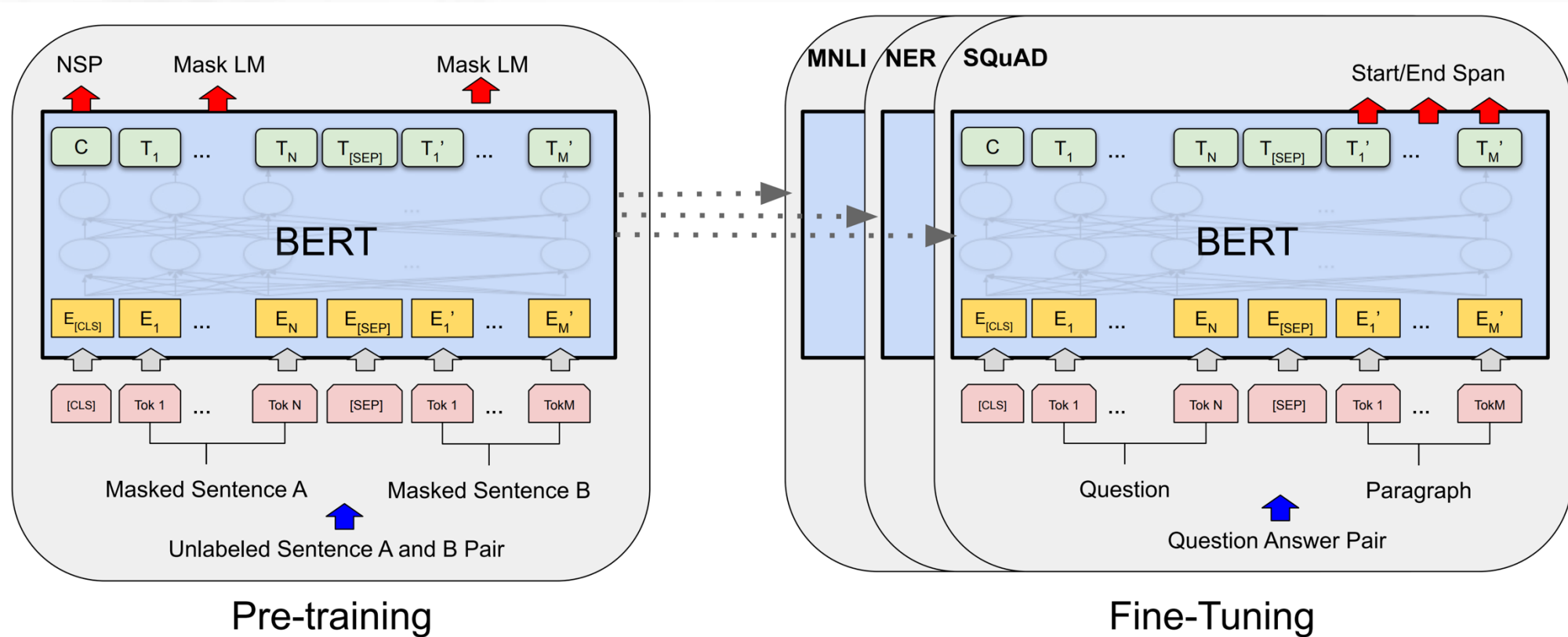
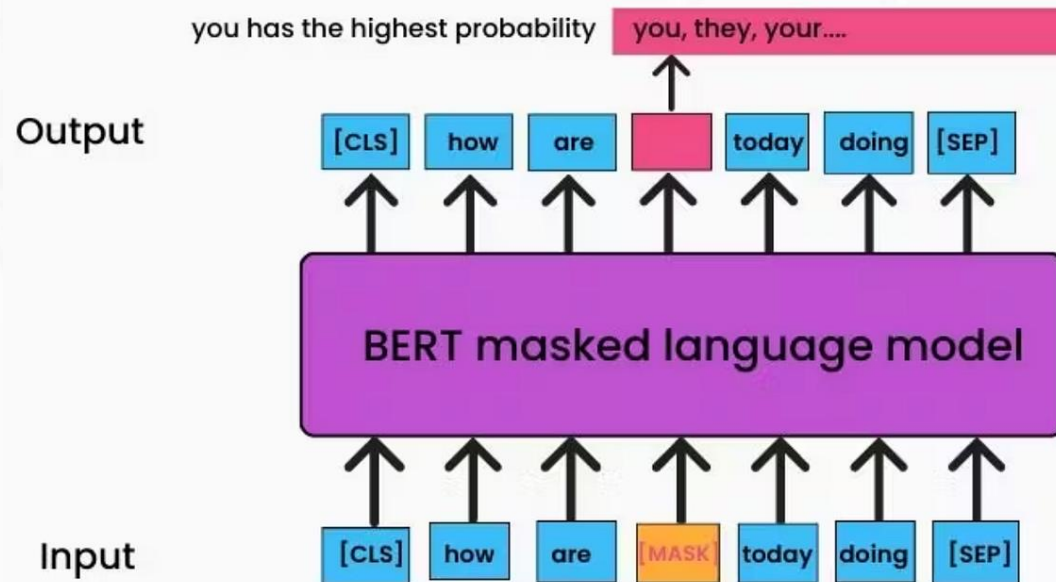


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. $[CLS]$ is a special symbol added in front of every input example, and $[SEP]$ is a special separator token (e.g. separating questions/answers).

BERT Pretraining

Masked Language Model

- The model randomly hides (masks) a percentage of words in a sentence and is trained to predict the missing words using surrounding context.
- This helps the model understand deep bidirectional relationships and grammar.



Next Sentence Prediction

- NSP acts as a binary classification task to understand sentence relationships. The model receives two sentences (Sentence A and Sentence B) with special [CLS] and [SEP] tokens used as boundaries.
- It then determines if Sentence B is the actual subsequent sentence in the original document.

Sentence 1	Sentence 2	Next Sentence?
I am going outside.	I will be back after 6.	YES
I am going outside.	You know nothing John snow.	NO

Seq-2-Seq Language Modelling

Google T5 - Scaling-Up Encoder-Decoder Model for Understanding + Generation

Text-to-Text Task

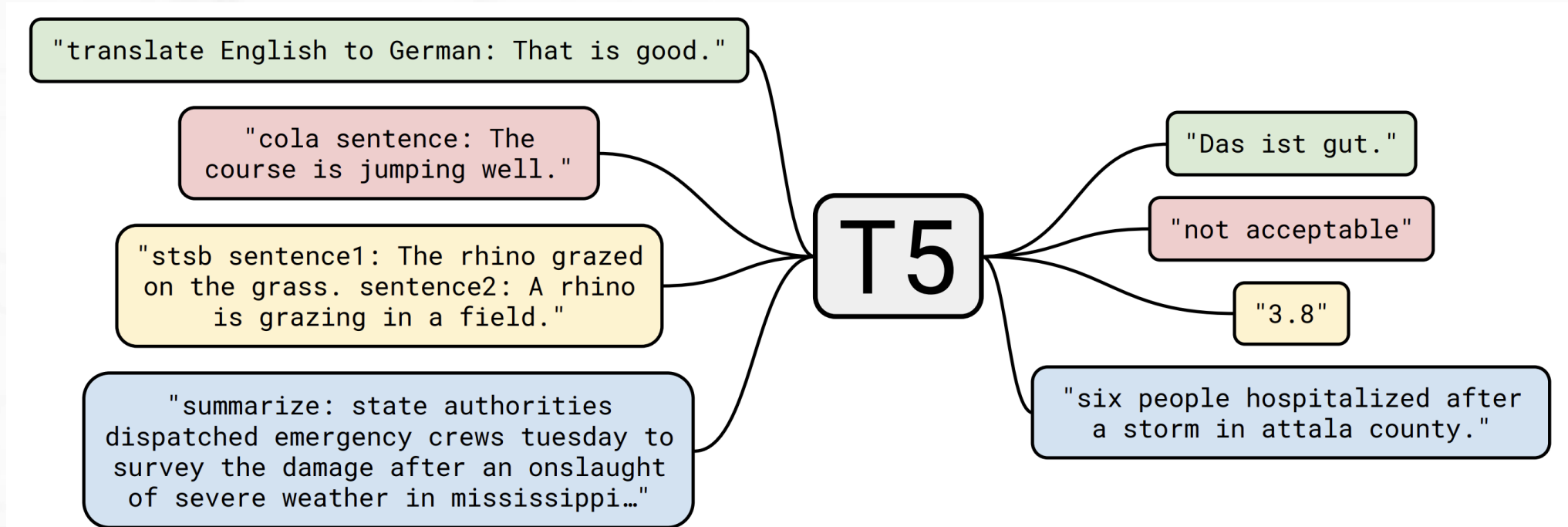


Figure 1: A diagram of our text-to-text framework. Every task we consider—including translation, question answering, and classification—is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks. It also provides a standard testbed for the methods included in our empirical survey. “T5” refers to our model, which we dub the “**Text-to-Text Transfer Transformer**”.

Architecture Choice & Training Strategy

